

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**Generative Neural Models for
Scalable Vector Graphics**

Master's Thesis

JOSEF KUCHARŮ

Brno, Spring 2026

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

Generative Neural Models for Scalable Vector Graphics

Master's Thesis

JOSEF KUCHAR

Advisor: Mgr. Michal Štefánik, Ph.D.

Department of Visual Computing

Brno, Spring 2026



Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

During the preparation of this thesis, I used the following AI tools:

- Grammarly for grammar checking,
- OpenAI Codex for code writing and text writing,
- Gemini Deep Research for finding relevant literature.

I declare that I used these tools in accordance with the principles of academic integrity. I checked the content and take full responsibility for it.

Josef Kuchař

Advisor: Mgr. Michal Štefánik, Ph.D.

Acknowledgements

I would like to thank my supervisor, Michal Štefánik, for his guidance and feedback throughout the work on this thesis. I am also grateful to my consultant, Marek Kadlčík, for helpful discussions and practical advice. Finally, I would like to thank my family for their support.

Abstract

Scalable vector graphics are a natural target format for generated visual content because they are resolution independent, compact, and editable. However, direct text-to-SVG generation remains less mature than raster image generation, where the strongest text-to-image models operate. This thesis studies a two-stage text-to-SVG pipeline that separates semantic image generation from raster-to-vector conversion.

The first stage adapts a pretrained text-to-image model with prompt conditioning and LoRA fine-tuning to generate SVG-like raster images. The second stage converts these images into a structured representation of cubic Bezier curves using a conditional flow-matching vectorizer trained on supervised raster-vector pairs, including procedurally generated synthetic data. This decomposition reuses the semantic strength of large raster generators while training a smaller model specifically for vector geometry.

Experiments support this decomposition at both the vectorization and end-to-end levels. The 0.26B-parameter vectorizer is competitive with neural SVG systems that are 4–31 times larger, and flow matching outperforms an autoregressive variant of comparable size. On generated rasters, it produces valid, compact SVGs more reliably than the StarVector baselines, while remaining less pixel-faithful than classical tracing. In qualitative end-to-end examples, the full pipeline outperforms the tested direct OmniSVG baselines by producing more complete and semantically aligned SVGs.

Keywords

scalable vector graphics, text-to-vector generation, raster-to-vector conversion, vector graphics generation, editable image representation, geometric reconstruction, neural vectorization, neural networks

Contents

Introduction	1
1 Background and Related Work	4
1.1 Scalable vector graphics	4
1.2 Bezier curves	5
1.3 Text-to-SVG generation	5
1.3.1 StarVector	6
1.3.2 OmniSVG	7
1.4 Image-to-SVG and vectorization methods	8
1.5 Text-to-image models adapted for vector graphics	10
1.6 Position of this work	11
2 Problem Formulation and Data	12
2.1 Task decomposition	12
2.2 Source SVG dataset	13
2.3 Alternative formulation	15
3 Stage 1: Raster Generation	16
3.1 Model adaptation	16
3.2 Training data and LoRA procedure	18
3.3 Evaluation	18
4 Stage 2: Vectorization	24
4.1 Bezier representation	24
4.2 SVG conversion to Bezier representation	27
4.2.1 Converted Bezier dataset	29
4.3 Synthetic data generator	30
4.4 Model architecture	33
4.4.1 Training objective	34
4.4.2 Network architecture	35
4.5 Training schedule	38
4.5.1 Synthetic pretraining	38
4.5.2 Fine-tuning on the SVG dataset	39
4.6 Evaluation	39
4.6.1 Architecture ablations	40
4.6.2 Synthetic pretraining results	42

4.6.3	Fine-tuning results	44
4.6.4	Vectorization benchmark on controlled inputs . . .	45
5	End-to-End Pipeline Evaluation	53
5.1	End-to-end text-to-SVG comparison	56
	Conclusion	58
5.2	Limitations	59
5.3	Future work	59
	Bibliography	61
A	Implementation artifacts	67

List of Tables

3.1	Stage 1 LoRA experiment comparing model quality across different LoRA ranks and training steps.	20
3.2	Qualitative Stage 1 comparison of text-to-raster model variants alongside existing text-to-SVG models.	22
3.3	Stage 1 benchmark of text-to-raster model variants. For comparison, the metrics are also reported for the existing text-to-SVG model OmniSVG.	23
4.1	Qualitative samples from the synthetic pretraining checkpoint.	44
4.2	Qualitative comparison on SVG validation samples. Empty cells indicate invalid output.	47
4.3	Qualitative comparison on synthetic-generator samples. Empty cells indicate invalid output.	48
4.4	Vectorization fidelity on SVG validation samples.	49
4.5	Vectorization fidelity on synthetic-generator samples.	50
4.6	SVG validity and complexity on SVG validation samples.	51
4.7	SVG validity and complexity on synthetic-generator samples.	51
5.1	Qualitative comparison of vectorizers on rasters generated by the selected Stage 1 model. The reference row contains the input rasters from the Z-Image Base prefixed + LoRA configuration from Section 3.3. Empty cells indicate invalid output.	54
5.2	Vectorization fidelity on generated raster images.	55
5.3	SVG validity and complexity on generated raster images.	55
5.4	End-to-end text-to-SVG examples generated by the proposed two-stage pipeline and by direct OmniSVG baselines.	57

List of Figures

1.1	A simple SVG document and its rendered output.	4
1.2	Rendered SVG path with the four points of a cubic Bezier segment.	5
1.3	High-level structure of the proposed text-to-vector pipeline. The second stage can be implemented either by a classical vectorization algorithm or by a learned vectorization model.	11
2.1	Examples of rasterized images from the SVG Repo-derived dataset.	14
2.2	Direct adaptation of raster-generation weights to Bezier prediction.	15
4.1	Examples of images produced by the synthetic data generator.	31
4.2	High-level structure of the proposed conditional flow-matching vectorizer.	34
4.3	Flow-matching and autoregressive vectorizer comparison.	40
4.4	Image reconstruction error with encoded and raw raster input.	41
4.5	Training loss with encoded and raw raster input.	42
4.6	Synthetic pretraining loss of the raster-to-vector model. . .	43
4.7	Image reconstruction error during synthetic pretraining. .	43
4.8	Fine-tuning loss of the raster-to-vector model on SVG Repo data.	45
4.9	Image reconstruction error during fine-tuning on SVG Repo data.	45

Introduction

Scalable Vector Graphics (SVG) is a common format for visual assets such as icons, logos, illustrations, diagrams, and interface elements. These assets often need to be resized, recolored, adjusted to a design system, or embedded in different documents and applications. Unlike raster images, which store a fixed grid of pixels, vector graphics describe images through geometric primitives and styling attributes [1]. This representation makes SVG images *resolution independent, compact, and editable*, and therefore suitable as a target format for generated visual content.

Modern text-to-image models have made it increasingly easy to create images from textual descriptions. This progress builds on diffusion probabilistic models and includes latent-diffusion text-to-image systems and recent transformer-based diffusion generators [2–6]. However, their most mature and widely available form is *raster image generation*. Such models can capture the semantic content of a prompt and produce visually rich results, yet the output is still a bitmap. For many practical uses, this bitmap is only an approximation of the *desired editable artifact*: it can be displayed, but it does not directly provide the structured paths, colors, and shapes that a designer or downstream program can manipulate as an SVG document.

This gap motivates the central problem of the thesis: *generating vector graphics from textual input*. Direct text-to-vector generation is difficult because the model must simultaneously learn semantic grounding, visual composition, geometric structure, and the syntactic constraints of vector graphics [7,8]. A *two-stage pipeline* offers a more modular alternative. First, a text prompt is converted into a raster image by a large pretrained generative model. Second, the raster image is translated into a vector representation composed of Bezier curves. This separation makes it possible to exploit the strengths of modern text-to-image models while developing a specialized vectorizer that operates in a well-defined geometric output space.

In this thesis, *vectorization* refers to raster-to-vector conversion: the problem of converting a pixel image into a visually similar vector graphic described by geometric primitives. Raster images represent

visual content as a discrete grid of colored pixels, whereas vector graphics describe shapes by parameters such as paths, curves, fills, and strokes. Rendering maps a vector description to pixels; vectorization attempts the inverse direction by recovering a compact and editable geometric description from raster evidence [9]. This inverse problem is inherently ambiguous, because many different sets of curves and shapes can render to nearly identical pixel images. A useful vectorizer therefore must balance image fidelity with structural simplicity, semantic editability, and validity of the resulting SVG [10].

A second reason for this separation is the *scale of available supervision*. Public text-to-image systems are trained in data regimes that are far beyond the available SVG corpus. For example, Stable Diffusion v1 was trained using large LAION image-text datasets, while LAION-5B contains billions of CLIP-filtered image-text pairs [11,12]. Building an analogous corpus for direct text-to-vector training would require not only many SVG files, but also reliable textual descriptions aligned with their vector structure. Such data is not available at a comparable scale for this thesis. This data imbalance motivates the decomposition used in this work: text-conditioned generation is delegated to a large pretrained raster model, while the custom model is trained for raster-to-vector conversion, where supervised synthetic examples can be generated by construction.

The objective of the thesis is to design, implement, and evaluate a modular text-to-vector pipeline that connects pretrained raster generation with a specialized neural vectorizer. The work is organized around four research questions:

- Can a pretrained text-to-image model be adapted to produce raster images that are easier to vectorize?
- Does procedurally generated Bezier data provide a useful pretraining signal for raster-to-vector generation?
- How does a conditional flow-matching vectorizer compare with autoregressive, classical tracing, and large neural SVG-generation baselines?
- What trade-offs arise between raster fidelity, SVG validity, compactness, and practical editability?

To answer these questions, the thesis develops the components needed for an end-to-end text-to-SVG pipeline: a training procedure for adapting a pretrained text-to-image model to an SVG-like raster domain, an SVG normalization procedure, a fixed-size Bezier representation, a synthetic Bezier data generator, and a conditional flow-matching vectorizer. The resulting pipeline is evaluated in terms of visual fidelity, SVG validity, compactness, and practical editability.

The thesis first reviews related work, then formulates the task and data pipeline. The main technical chapters then follow the two stages directly: raster generation and vectorization are each described together with their training setup and evaluation. The final evaluation studies the complete pipeline before the thesis summarizes the achieved results and limitations.

1 Background and Related Work

This thesis is related to several research directions at the intersection of generative modeling, vector graphics, and multimodal learning. The most relevant prior work can be grouped into the following categories.

1.1 Scalable vector graphics

Scalable Vector Graphics (SVG) is an XML-based format for describing two-dimensional vector images. Unlike raster images, which store a fixed grid of pixels, an SVG document stores graphical elements such as paths, rectangles, circles, gradients, fills, and strokes. The image is produced only when this description is rendered, which makes SVG resolution independent and suitable for icons, illustrations, diagrams, and other graphics that should remain editable after creation [1].

The central primitive for this work is the SVG path element. A path stores a sequence of drawing commands in its `d` attribute. For example, the command `M` moves the current point and the command `C` draws a cubic curve from the current point to a new endpoint using two intermediate control points [13]. Figure 1.1 shows a minimal SVG document with three primitives and its rendered output.

```
<svg viewBox="0 0 120 80"
  xmlns="http://www.w3.org/2000/svg">
  <rect x="12" y="14" width="32" height="32"
    fill="#f97316" />
  <circle cx="80" cy="30" r="18"
    fill="#22c55e" />
  <path d="M 18 66 C 42 44, 76 86, 104 58"
    fill="none" stroke="#2563eb"
    stroke-width="6" stroke-linecap="round" />
</svg>
```

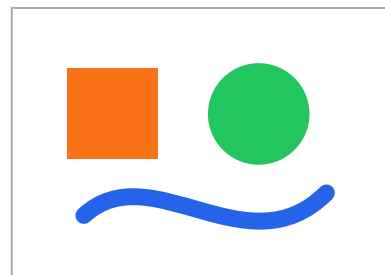


Figure 1.1: A simple SVG document and its rendered output.

1.2 Bezier curves

The Bezier curves used in this work are a standard way of representing curved SVG paths. In SVG path data, a cubic Bezier segment is specified by an endpoint and two control points relative to the current point; sequences of such segments can describe smooth contours, while fills and strokes determine how the paths are rendered [13]. The control-point geometry of a cubic Bezier segment is illustrated in Figure 1.2.

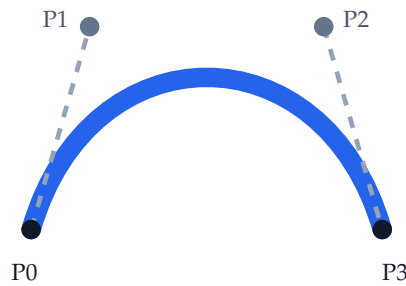


Figure 1.2: Rendered SVG path with the four points of a cubic Bezier segment.

1.3 Text-to-SVG generation

The closest line of work aims to generate SVG content directly from textual descriptions. These approaches typically formulate the problem either as code generation, where the model predicts SVG tokens or commands autoregressively, or as structured graphics generation, where the model predicts vector objects and their attributes in a more constrained representation [7,8]. The main advantage of direct text-to-SVG generation is that it avoids an intermediate raster representation and can therefore produce editable vector output in a single stage.

However, this single-stage formulation combines two different tasks. The model must first understand what the text prompt describes, such as which objects should appear and how they relate

to each other. It must then express this drawing as a valid SVG document, where visual elements are represented by precise coordinates, path commands, attributes, and ordering decisions.

This makes SVG generation difficult not only because the output is visual, but also because the output is structured code. The model has to produce valid path data, assign attributes such as fill color and opacity, decide how shapes are layered, and handle the ambiguity that multiple SVG programs can render to very similar raster images. Recent SVG-generation systems therefore either generate full SVG code with a code-oriented language model [7] or simplify SVGs by removing most SVG attributes and restricting paths to a small set of basic draw commands before training [8].

From the perspective of this thesis, direct text-to-SVG methods are important as a conceptual baseline. They address the same end goal as the proposed system, but differ in where the complexity is handled. In the direct setting, semantic generation and vector-structure generation are solved simultaneously. In the present work, these two difficulties are separated into a raster generation stage and a dedicated vectorization stage.

Representative recent examples include StarVector [7] and OmniSVG [8]. Both systems treat SVG generation as a sequence modeling problem, but they make different choices about the output representation. StarVector predicts native SVG markup, whereas OmniSVG normalizes SVGs into tokenized atomic path and fill commands. The two systems also differ in how they connect visual or textual conditioning to the generated vector representation.

1.3.1 StarVector

StarVector [7] formulates SVG synthesis as multimodal code generation. The model is conditioned either on a raster image or on a text instruction and then predicts the SVG document autoregressively as code. For image-to-SVG generation, the raster input is encoded by a vision transformer, projected through an adapter into the language-model embedding space, and prepended as visual tokens before the SVG token sequence. For text-to-SVG generation, the conditioning signal is provided by the language model’s ordinary text tokenizer. In both cases, the decoder is trained with a next-token objective over

SVG code, so inference amounts to sampling SVG markup until an end-of-SVG token is produced.

A central design choice in StarVector is to operate in the native SVG code space. This allows the model to use higher-level primitives such as circles, ellipses, polygons, text, and styling constructs when they are appropriate. The motivation is that a semantically recognized circle should be emitted as a compact SVG primitive rather than approximated by many small Bezier path segments. This distinguishes StarVector from classical vectorizers, which often optimize pixel fidelity but can produce long, fragmented paths with limited semantic editability.

The training data for StarVector are collected in SVG-Stack, a large dataset of approximately two million SVG samples paired with raster renderings and synthetic text descriptions. The dataset is intended to cover a broad range of web SVG syntax and primitives, which is important because SVG generation is not only a geometric task, but also requires the model to produce syntactically valid SVG markup. StarVector is evaluated with SVG-Bench on image-to-SVG, text-to-SVG, and diagram-generation tasks [7].

1.3.2 OmniSVG

OmniSVG [8] also uses an autoregressive formulation, but it avoids generating raw XML markup directly. Instead, the input SVGs are simplified into a sequence of atomic drawing commands and attributes. The representation includes move, line, cubic Bezier, elliptical arc, close-path, and fill commands, while coordinates and command types are discretized into tokens. This tokenizer places vector geometry into the same sequential modeling framework as text and image tokens, but it removes much of the syntactic variability of full SVG XML. For example, the same filled rectangle can be written as a `rect` element, as a path with line and close-path commands, inside a group that inherits its fill color, or under a coordinate transform. A normalized command sequence reduces these alternatives to explicit drawing operations and attributes. The model is built on a pre-trained vision-language model, Qwen2.5-VL [14], and uses text and image inputs as prefix tokens before generating the SVG command sequence with a next-token prediction objective.

The purpose of this parameterization is to separate the higher-level structure of the drawing from low-level coordinate prediction. Raw SVG code contains many equivalent ways to express the same image, for example through transforms, groups, or different primitive forms. OmniSVG reduces this ambiguity by normalizing SVGs with tools such as `picosvg` [15] and representing them with a limited set of atomic commands.

OmniSVG is trained and evaluated with MMSVG-2M, a multi-modal dataset containing about two million SVG assets, including icons, illustrations, and more complex character graphics. Its benchmark covers text-to-SVG, image-to-SVG, and character-reference SVG generation [8]. This makes OmniSVG a useful reference point for scalable conditional SVG generation: it demonstrates that large vision-language models can be adapted to produce detailed editable vector outputs when a sufficiently standardized SVG tokenizer and large-scale data are available.

Both StarVector and OmniSVG are closely aligned with the overall objective of this thesis, but they solve the problem in a different place in the pipeline. They aim to learn semantic generation and vector-structure generation jointly inside a single autoregressive model. The method developed here deliberately decomposes the task into raster generation followed by raster-to-vector conversion. In this setup, the vectorizer is not trained to interpret text or choose SVG structure from a prompt. Its task is limited to reconstructing vector geometry from an image, so it can be trained on synthetic raster-vector pairs generated from known shapes rather than on SVG examples annotated with natural-language captions.

1.4 Image-to-SVG and vectorization methods

A second related area consists of methods that convert raster images into vector graphics. Classical vectorization systems rely on contour extraction, polygon fitting, spline fitting, or optimization-based refinement. More recent learning-based approaches predict vector primitives directly from raster input, often using transformers, diffusion models, or autoregressive decoders. These methods are highly

relevant to the second stage of the proposed pipeline, because they focus on the structured reconstruction problem independently of text conditioning.

The model proposed in this thesis belongs primarily to this category, but it differs in the use of flow matching [16] and in the specific Bezier-segment representation employed for training and decoding.

Flow matching is a generative modeling objective for learning a continuous vector field that transports samples from a simple source distribution, such as Gaussian noise, toward the data distribution. The original flow matching work frames this as simulation-free training of continuous normalizing flows by regressing vector fields along prescribed probability paths, with diffusion paths included as a special case [16]. In this thesis, the same idea is applied in the continuous space of Bezier-segment tensors: the model learns how a noisy vector representation should move toward a valid raster-conditioned vector graphic.

Representative approaches include differentiable vector-graphics rasterization for optimization and learning [17] and learned SVG representations such as DeepSVG [18].

These works motivate parts of the problem formulation, but they are not direct baselines for the evaluation used in this thesis. DiffVG supplies a differentiable rasterizer and optimization substrate; turning it into a comparable method would require specifying an additional image-fitting pipeline, including the primitive budget, initialization, losses, and optimization schedule. DeepSVG, in turn, studies generative modeling of existing SVG sequences rather than raster-conditioned vectorization. The empirical evaluation therefore compares selected systems that can be run as end-to-end image-to-SVG methods on the same input set.

Existing vectorizers also serve as empirical baselines. Classical systems such as Potrace [9] and vtracer [19] are strong engineering tools, but they typically optimize local image fidelity and often produce dense, fragmented paths when the input contains noise, compression artifacts, blur, or soft color transitions. The experiments use vtracer as the classical tracing baseline. Recent neural text-to-SVG systems, by contrast, often rely on large vision-language models fine-tuned on SVG data [7,8].

1.5 Text-to-image models adapted for vector graphics

Another important line of prior work concerns large text-to-image models that are adapted to generate images in a style suitable for graphic design, illustration, icons, or symbol-like imagery. Even when such models do not produce vector output directly, they can already turn text prompts into images with the requested objects, attributes, and spatial arrangement. This idea motivates the first stage of the proposed pipeline, where a pretrained text-to-image model is adapted and used as a controllable raster generator.

Modern text-to-image systems are commonly based on diffusion models. A diffusion model learns to reverse a gradual noising process: during training, clean images are corrupted with noise, and the network learns a denoising transition that can be applied iteratively to synthesize new samples from noise [2]. Latent diffusion models make this process more efficient by performing the denoising in a compressed latent image space rather than directly in pixel space, which is one reason they became practical for high-resolution conditional image synthesis [3].

This line of work makes the decomposition adopted in this thesis plausible, but it does not by itself provide a raster generator specialized for subsequent SVG reconstruction. The first stage of this thesis therefore adapts a pretrained diffusion model toward simplified, illustration-like images that are easier to vectorize. This allows the semantic burden of text understanding to remain with the raster generator, while the second stage can focus on geometric reconstruction.

Prior work on text-to-image customization shows that pretrained generators can be adapted to narrower visual concepts or styles without training a new model from scratch. Examples include subject-driven fine-tuning with DreamBooth [20], parameter-efficient multi-concept adaptation in Custom Diffusion [21], and style transfer through StyleDrop [22]. LoRA provides the parameter-efficient adaptation mechanism used in this thesis [23].

1.6 Position of this work

The proposed method is a two-stage text-to-vector pipeline. The first stage uses an adapted text-to-image model to synthesize a raster image from a text prompt. The second stage converts this image to SVG, either with a classical vectorization algorithm or with the learned vectorization model developed in this thesis.

The raster image is the interface between the two stages. This separates text understanding and visual composition from the geometric problem of producing editable SVG paths. The method therefore uses text-to-image generation as a source of vectorization-friendly inputs, while leaving the final SVG conversion to a dedicated vectorization stage. The high-level pipeline is shown in Figure 1.3.

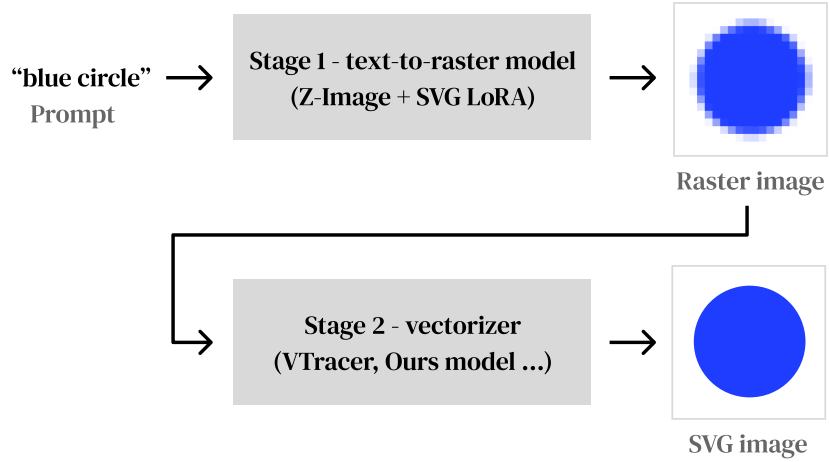


Figure 1.3: High-level structure of the proposed text-to-vector pipeline. The second stage can be implemented either by a classical vectorization algorithm or by a learned vectorization model.

2 Problem Formulation and Data

This chapter defines the decomposition used by the thesis, describes the shared data source used by both stages of the pipeline, and discusses the main alternative formulation considered during the work. It separates the question of what is being solved from the implementation details described in the stage-specific chapters.

2.1 Task decomposition

The proposed system consists of the following two stages:

- Stage 1: text-to-raster generation. A text-conditioned raster generation model produces bitmap images with characteristics suitable for subsequent vector graphics generation.
- Stage 2: raster-to-vector generation. The raster image can be converted to SVG by any vectorization method, for example a classical algorithm such as *vttracer* or the custom conditional flow-matching model developed in this thesis. The learned model is trained from scratch to predict a sequence of Bezier-segment descriptors, which can then be decoded into SVG paths.

From a methodological perspective, the first stage addresses semantic image synthesis from text, while the second stage addresses structured geometric reconstruction. The interface between the two stages is the raster image itself, which allows the vectorization model to be trained independently of the text-to-image model once a suitable image distribution has been established.

Training a direct text-to-Bezier model would require large quantities of paired text descriptions and vector annotations. This is a much stronger data requirement than ordinary raster-to-vector supervision: the model would need to learn both language-conditioned visual composition and precise vector geometry from the same paired examples. In contrast, public text-to-image systems rely on web-scale image-text corpora: Stable Diffusion v1 is documented as being trained on LAION-5B subsets, and LAION-5B contains 5.85 billion CLIP-filtered image-text pairs [11,12]. The SVG dataset used

in this thesis contains approximately 216k examples before filtering and conversion losses, and its captions are automatically generated rather than a web-scale human language supervision signal. The vectorizer-based decomposition avoids this bottleneck: the raster-to-vector model can be pretrained on procedurally generated synthetic data, for which vector labels are available by construction, and then adapted to real SVG data. The central experimental question is whether sufficiently varied synthetic pretraining can transfer to real vector graphics after fine-tuning.

This is a practical advantage of the two-stage formulation over direct text-to-vector generation. For the vectorizer, every procedurally generated vector scene can be rendered to a raster image and used immediately as a paired training example. This makes it possible to create an effectively unlimited amount of supervised data at low cost. In contrast, direct text-to-SVG training would require SVGs paired with high-quality textual descriptions, which are much harder to collect at scale and are not produced automatically by the vector representation itself.

2.2 Source SVG dataset

The two-stage formulation requires a source collection that can support both semantic supervision and geometric supervision. For Stage 1, SVG files must be paired with text so that the raster generator can be adapted to prompts. For Stage 2, the same graphical content must be available as vector markup so that it can be converted into Bezier targets. A dataset with both SVG code and associated captions is therefore useful as a shared source for the whole pipeline.

Both stages use the `mikronai/svg-svgrepo` dataset distributed through Hugging Face [24] as this source SVG collection. The dataset is derived from SVG Repo graphics [25] and is provided as a tabular Parquet dataset. At the time of use, the default subset contained approximately 216k examples, split into approximately 214k training examples, 1010 validation examples, and 1010 test examples. Each row contains the raw SVG markup in the `item_svg` field, collection

and item identifiers, license metadata, item tags, an item title, and four generated text captions with associated generation metadata.

In the proposed pipeline, the SVG files are rasterized and paired with textual captions for LoRA adaptation of the text-to-raster model. The same source SVG files are also converted into the internal Bezier representation described later in this chapter. The original SVG collection is therefore used as a source of semantic supervision for raster generation and as the source material for geometric supervision in raster-to-vector learning.

The dataset is heterogeneous because it aggregates graphics from many original collections and licenses. This diversity is useful for evaluating generalization, but it also requires filtering and normalization before training. In particular, SVGs containing unsupported constructs such as gradients, masks, or embedded style blocks are excluded, while supported geometric content is normalized by the preprocessing pipeline described below. Examples from the rasterized dataset are shown in Figure 2.1.



Figure 2.1: Examples of rasterized images from the SVG Repo-derived dataset.

2.3 Alternative formulation

The main alternative to the two-stage decomposition is to adapt a pre-trained text-to-raster model directly into a text-to-bezier model. This would be conceptually attractive, because it would collapse the whole pipeline into one model while preserving the semantic knowledge of the pretrained generator. An experiment with this approach was performed by comparing a model initialized from pretrained text-to-raster weights with a model whose parameters were reset before training. The resulting optimization curves are shown in Figure 2.2. Over the shared training interval, the reset model learns faster and reaches a lower training loss than the model initialized from raster-generation weights. This indicates that the pretrained weights do not provide a useful initialization for Bezier prediction in this setting.

Although the original model has learned a strong representation for raster image generation, that task requires a substantially different internal representation from the one needed to predict structured Bezier control points and attributes. The approach is therefore not used as the main method, although it remains a possible large-scale direction if substantially more paired text-vector data become available.

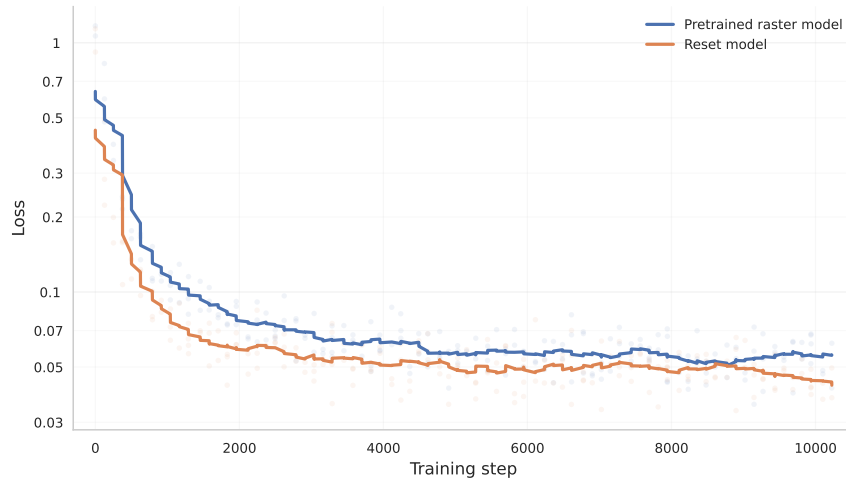


Figure 2.2: Direct adaptation of raster-generation weights to Bezier prediction.

3 Stage 1: Raster Generation

The first stage must provide the vectorizer with images that preserve the semantic content of the text prompt while remaining simple enough to be approximated by vector paths. This creates a different target from general photorealistic image generation: texture, clutter, and ambiguous foreground boundaries can make the second stage harder even when the raster image is visually plausible. The chapter therefore describes how the text-to-raster model is adapted toward a vector-like visual domain, how the LoRA training is performed, and how the final raster-generation configuration is selected.

3.1 Model adaptation

Training a competitive text-to-image model from scratch is not required for the proposed decomposition, because the semantic and compositional knowledge needed to interpret text prompts is already present in large pretrained raster generators. The adaptation problem is narrower: the model should retain this knowledge while shifting its outputs toward simplified composition, cleaner silhouettes, reduced texture complexity, and visual styles that are easier to approximate by Bezier curves.

For this reason, the first stage is based on the pretrained Z-Image family of image-generation models [6] and adapts it to the target visual domain using paired image-text examples. The goal of this adaptation is to make the model produce raster outputs with the desired vector-like properties while retaining the semantic coverage of the pretrained generator.

The raster generator must also be practical to use repeatedly, because it is needed for qualitative inspection, ablation experiments, and the final end-to-end demonstration. Z-Image provides a comparatively compact starting point for these experiments: the Z-Image paper describes a 6B-parameter architecture and reports performance competitive with selected contemporary open models with

much larger parameter counts, including Qwen-Image (20B) [5], FLUX.2 (32B) [26], and HunyuanImage 3.0 (80B) [27,6].

The accelerated Turbo variant has the same model scale, but reduces the number of diffusion evaluations, which lowers the cost of repeated raster generation during dataset construction, qualitative inspection, and ablation experiments. Finally, prompt adherence was a central selection criterion. The reported evaluations include human-preference rankings and benchmarks for object-centric generation, dense prompt following, and instruction following [6]. This is important because the vectorization stage operates only on the generated raster.

The adaptation method should preserve the broad semantic and compositional knowledge of the pretrained text-to-image system while changing only a comparatively small number of parameters. LoRA provides this parameter-efficient fine-tuning mechanism. Instead of updating all weights of a large pretrained model, it freezes the base weights and learns small trainable low-rank matrices whose product approximates the desired weight update [23]. Because the original weights are not overwritten, this also reduces the risk of forgetting the pretrained model’s general prompt-following and image-generation capabilities, consistent with findings on parameter-efficient fine-tuning in vision transformers [28].

For inference, the base Z-Image model and the accelerated Z-Image Turbo model were evaluated with different sampling settings. The base model was sampled with 50 denoising steps and classifier-free guidance [29] scale 4. By contrast, Z-Image Turbo was sampled with 8 denoising steps and without classifier-free guidance, because the turbo model is guidance-distilled and is intended to operate without an explicit CFG term at inference time [30].

Classifier-free guidance is a conditioning technique for diffusion models in which the model is trained with both conditional and unconditional inputs. At sampling time, the conditional prediction is strengthened by comparing it with the unconditional prediction, and the guidance scale controls how strongly the sample is pushed toward the prompt [29]. This usually improves prompt adherence but requires additional model evaluations unless the effect has been distilled into a faster model.

After training, the learned LoRA weights are loaded into the Z-Image Turbo pipeline for fast sampling. This design preserves the knowledge of the original pretrained model while making inference substantially more efficient than full base-model fine-tuning. The use of the same SVG-style LoRA on the distilled turbo variant is motivated by the observation that distilled diffusion models can preserve the controllability of their teacher models, allowing controls learned for the base model to remain useful after distillation [31]. In the experiments reported below, prompts are prefixed with “SVG illustration with white background. ” to bias the generator toward clean foreground graphics on a simple canvas. The resulting samples are then assessed both as images and as inputs for downstream vectorization.

3.2 Training data and LoRA procedure

The Stage 1 training setup is intentionally simple, because the main question is whether a lightweight domain adaptation improves raster outputs for downstream vectorization rather than whether an extensive fine-tuning recipe can be optimized. The LoRA adaptation was trained using the AI-Toolkit framework [32] with the AdamW optimizer [33] and a learning rate of 1×10^{-4} . This configuration was used as the default starting point for the Stage 1 adaptation experiments. The LoRA rank and checkpoint selection are evaluated in Section 3.3.

3.3 Evaluation

Both CLIP and DINO scores are computed as cosine similarities between normalized feature vectors

$$\text{sim}(a, b) = \frac{a \cdot b}{\|a\|_2 \|b\|_2}$$

CLIP-based similarity uses a joint image-text representation learned from natural-language supervision and is therefore useful for measuring semantic alignment [34]. DINO-based similarity uses self-supervised visual features intended to transfer across image distributions and tasks [35]. Both types of feature-space similarity are also relevant because they have been reported to correlate well with human preference in vector-graphics evaluation [7].

The vectorization MSE measures how well a raster image can be vectorized. For each generated raster image, the benchmark first converts the image to SVG using *vt racer*, an open-source raster-to-vector converter that traces color raster images into SVG paths [19], with its default command-line settings. The resulting SVG is then rasterized back to the original image resolution on a white background. The score is the mean squared error between the original generated RGB image and this rerendered image,

$$\frac{1}{3HW} \sum_{c,y,x} (I_{c,y,x} - \hat{I}_{c,y,x})^2$$

where pixel values are measured in the usual 0–255 RGB range. This metric does not compare the generated image to the reference image directly.

Instead, it measures how much visual information is lost when the image is approximated by a standard vectorization tool, so lower values indicate images whose shapes and colors are easier to represent as clean vector graphics.

This metric should not be interpreted as a complete measure of vector quality. As noted in prior vectorization work, a low pixel error can still correspond to an overly complex SVG with many redundant paths, while a compact and easy-to-edit SVG may differ slightly at the pixel level [9,7]. The metric is therefore used here as a practical proxy for traceability, not as a replacement for evaluating path count, primitive structure, or editability.

3. STAGE 1: RASTER GENERATION

Time steps	Metric	LoRA Rank		
		4	16	64
500	CLIP similarity ↑	0.880	0.872	0.885
	DINO similarity ↑	0.607	0.599	0.599
	Vectorization MSE ↓	299.456	187.121	205.943
1000	CLIP similarity ↑	0.886	0.887	0.885
	DINO similarity ↑	0.616	0.621	0.620
	Vectorization MSE ↓	328.455	128.576	199.657
1500	CLIP similarity ↑	0.886	0.887	0.884
	DINO similarity ↑	0.622	0.618	0.615
	Vectorization MSE ↓	206.473	143.796	346.287
2000	CLIP similarity ↑	0.885	0.885	0.887
	DINO similarity ↑	0.627	0.614	0.625
	Vectorization MSE ↓	142.765	97.675	168.836
2500	CLIP similarity ↑	0.888	0.888	0.882
	DINO similarity ↑	0.626	0.627	0.620
	Vectorization MSE ↓	174.613	92.145	273.638
3000	CLIP similarity ↑	0.887	0.887	0.888
	DINO similarity ↑	0.626	0.635	0.632
	Vectorization MSE ↓	173.293	92.112	132.058
3500	CLIP similarity ↑	0.886	0.887	0.888
	DINO similarity ↑	0.626	0.630	0.633
	Vectorization MSE ↓	166.197	93.775	157.162
5000	CLIP similarity ↑	0.886	0.886	0.883
	DINO similarity ↑	0.620	0.631	0.617
	Vectorization MSE ↓	178.814	166.223	172.532

Table 3.1: Stage 1 LoRA experiment comparing model quality across different LoRA ranks and training steps.

The LoRA adaptation was first evaluated separately in order to select a rank and checkpoint for the Stage 1 comparison. Three LoRA ranks, namely 4, 16, and 64, were evaluated at checkpoints saved every 500 steps. The resulting CLIP similarity, DINO similarity, and vectorization MSE values are summarized in Table 3.1. Due to computational constraints, the experiment was evaluated on a subset of 100 validation samples rather than on the full validation set of 1010 samples. The explored grid of ranks and checkpoints is therefore intentionally coarse.

A more fine-grained sweep over ranks, training durations, and sampling seeds would provide a more precise model-selection criterion, but was outside the available compute budget.

Based on the rank and checkpoint experiment, the LoRA model with rank 16 at 3000 training steps was selected for subsequent Stage 1 experiments. This checkpoint achieves the highest DINO similarity and the lowest vectorization MSE in the experiment. Although its CLIP similarity is slightly lower than the best observed value, the difference is small, making this checkpoint a reasonable choice for subsequent experiments.

The selected LoRA checkpoint is then used in the broader Stage 1 comparison. The comparison of several Stage 1 variants is shown qualitatively in Table 3.2 and quantitatively in Table 3.3. The compared variants include the base Z-Image model, prompt-prefixing strategies, the accelerated Z-Image-Turbo model, and the LoRA adaptation applied to both the base and turbo model variants. Higher CLIP and DINO similarity indicate better alignment with the reference images, whereas lower vectorization MSE indicates that the generated raster outputs are easier to convert in the second stage.

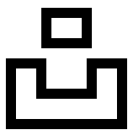
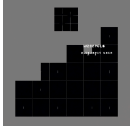
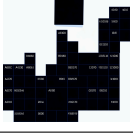
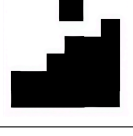
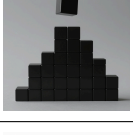
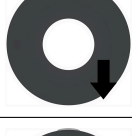
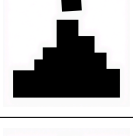
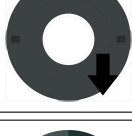
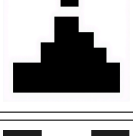
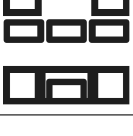
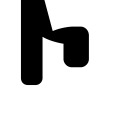
Reference				
Z-Image Base				
Z-Image Base prefixed				
Z-Image Base prefixed + LoRA				
Z-Image Turbo				
Z-Image Turbo prefixed				
Z-Image Turbo prefixed + LoRA				
OmniSVG1.1 8B				
OmniSVG1.1 4B				

Table 3.2: Qualitative Stage 1 comparison of text-to-raster model variants alongside existing text-to-SVG models.

3. STAGE 1: RASTER GENERATION

Variant	CLIP similarity $\uparrow$	DINO similarity $\uparrow$	Vectorization MSE $\downarrow$
Z-Image Base	0.818	0.509	266.565
Z-Image Base prefixed	0.820	0.546	230.160
Z-Image Base prefixed + LoRA	0.883	0.617	145.453
Z-Image Turbo	0.827	0.510	227.692
Z-Image Turbo prefixed	0.871	0.584	142.712
Z-Image Turbo prefixed + LoRA	0.879	0.600	143.175
OmniSVG1.1 8B	0.834	0.425	51.146
OmniSVG1.1 4B	0.828	0.391	57.621

Table 3.3: Stage 1 benchmark of text-to-raster model variants. For comparison, the metrics are also reported for the existing text-to-SVG model OmniSVG.

The results suggest that prompt prefixing, i.e. adding a fixed SVG-style instruction before each user prompt, has a substantial effect, especially for the turbo model. The best overall semantic similarity is obtained by the Base prefixed + LoRA configuration, while the lowest vectorization error is achieved by Turbo prefixed. The LoRA adaptation substantially improves both semantic similarity and vectorization MSE for the base model, but for the turbo model its effect is mostly limited to semantic similarity: the vectorization MSE remains nearly unchanged compared with Turbo prefixed.

The Base prefixed + LoRA configuration is therefore used as the reference text-to-raster setting in the final end-to-end evaluation, where image quality is prioritized. At the same time, Turbo prefixed + LoRA remains a reasonable practical alternative when inference speed is more important, because it preserves most of the semantic-similarity gain while using the accelerated turbo sampler.

4 Stage 2: Vectorization

The second stage addresses the part of the pipeline that cannot be delegated to a pretrained raster generator: recovering a compact, valid, and editable vector description from pixels. This task is ambiguous because the same raster image can be explained by many different sets of curves, colors, and path orderings. The stage therefore requires both a constrained output representation and a model trained specifically for raster-to-vector reconstruction. The following sections describe the Bezier representation, data preparation, synthetic data generation, and the architecture of the proposed vectorizer.

4.1 Bezier representation

The representation must satisfy two competing requirements. It should be close enough to SVG to reconstruct ordinary path geometry, but regular enough to serve as the output space of a neural generative model. A fixed-dimensional continuous descriptor for each segment is more suitable for this purpose than a heterogeneous command language containing separate primitives for lines, arcs, rectangles, circles, and paths. Converting all supported geometry to cubic Bezier segments therefore gives the model a single native output type while still preserving the ability to reconstruct standard SVG paths [13].

The vector output used throughout this work is therefore based on a hierarchical representation consisting of shapes, paths, and individual Bezier segments. A shape corresponds to one filled graphical object and is assigned a single RGB color and opacity value. Each shape contains one or more paths, and each path consists of a sequence of cubic Bezier curves. In the implementation, one curve is stored as a tuple $((x_0, y_0), (x_1, y_1), (x_2, y_2), (x_3, y_3))$, where (x_0, y_0) is the start point, (x_1, y_1) and (x_2, y_2) are the two control points, and (x_3, y_3) is the endpoint. This convention is used for geometric manipulation and SVG export.

This representation also covers common geometric primitives that are not originally specified as cubic curves. A straight line segment is a special case of a cubic Bezier curve where both control points lie on the segment between the endpoints, so the curve has no deviation from the line. Circular and elliptical arcs are represented by cubic arc approximations.

For a unit quarter circle from $(1, 0)$ to $(0, 1)$, the standard symmetric approximation uses control points $(1, \kappa)$ and $(\kappa, 1)$. Matching the midpoint at $t = \frac{1}{2}$ to the circular diagonal gives

$$\frac{1}{2} + 3\frac{\kappa}{8} = \frac{\sqrt{2}}{2}$$

and therefore

$$\kappa = \frac{4}{3}(\sqrt{2} - 1) \approx 0.5522847498$$

[36]. Scaling this construction along the horizontal and vertical axes gives the corresponding ellipse approximation.

For learning, the hierarchical SVG structure is converted into a flat sequence of segment descriptors. Each segment is represented by a 13-dimensional vector

$$s = (x_0, y_0, x_1, y_1, x_2, y_2, r, g, b, \alpha, f_p, f_s, f_r)$$

where (x_0, y_0) denotes the start point of the segment, (x_1, y_1) and (x_2, y_2) are the control points, (r, g, b) is the color, α is opacity, f_p indicates the beginning of a new SVG path element, f_s indicates the beginning of a new subpath within the current path, and f_r is a validity flag distinguishing real segments from padding.

The endpoint is omitted from the learned representation, because it is implied by the start point of the following segment. For the last segment in a path, the endpoint is defined by the start point of the first segment, which closes the contour explicitly.

This convention follows the sequential nature of SVG path data, where each command starts from the current point left by the previ-

ous command and updates that current point after execution [13]. Storing endpoints implicitly removes one redundant coordinate pair per segment, while still allowing the full path to be reconstructed when segment order and path boundaries are known.

All quantities are normalized to the interval $[-1, 1]$. Let the original raster image have width W and height H . Coordinates are normalized with respect to the image center

$$c_x = \frac{W}{2} \quad c_y = \frac{H}{2}$$

and the uniform scale factor

$$\lambda = \frac{2}{\max(W, H)}$$

The normalized coordinates are therefore

$$\tilde{x} = (x - c_x)\lambda, \quad \tilde{y} = (y - c_y)\lambda$$

This choice preserves aspect ratio and maps the larger image dimension to the full interval $[-1, 1]$.

Color channels originally stored in $[0, 255]$ are mapped linearly to $[-1, 1]$, and opacity values from $[0, 1]$ are mapped by the same affine transformation. Binary structural flags are represented as $+1$ for true and -1 for false, which keeps every output dimension on a common numerical scale.

Since the number of segments varies across examples, the model operates on a fixed-length tensor of shape $(N, 13)$, where N is the maximum number of segments allowed for one sample. If an SVG contains fewer than N segments, the remaining rows are padded with zeros and their validity flag is set to -1 . If an SVG contains more than N segments, the representation is truncated to the first N segments. The validity flag thus serves two purposes: it masks padded positions during learning and enables the decoder to ignore non-existent segments during reconstruction.

The inverse mapping reconstructs the hierarchical vector structure from the predicted tensor. First, all rows with $f_r l = 0$ are discarded. The remaining normalized coordinates, colors, and opacity values are denormalized back to image space and original attribute ranges.

The flags f_p and f_s are thresholded at zero and used to determine whether a segment starts a new shape or a new subpath. Because color and opacity are predicted per segment, the final attribute of a reconstructed shape is obtained by averaging these values over all its constituent segments. Finally, each path is closed by connecting the endpoint of every segment to the start point of the next one, with the last segment connected back to the first. This yields a compact sequence representation that is convenient for neural prediction while still preserving the topology required for valid SVG reconstruction.

4.2 SVG conversion to Bezier representation

Raw SVG files are too varied to be used directly by the fixed tensor representation. They may contain different primitive types, nested groups, transforms, strokes, and style attributes. Before these data can be used for training, each SVG must therefore be converted into a uniform representation compatible with the tensor encoding described above. The conversion procedure maps every supported graphical element to a collection of filled cubic Bezier paths together with a shared color and opacity.

The conversion begins with structural simplification. SVG files are first processed externally in Inkscape, a vector graphics editor [37]. During this stage, all objects are converted to paths and strokes are expanded into filled outlines. This step removes many forms of SVG variability and ensures that the subsequent parser operates on explicit geometric contours rather than on higher-level drawing commands. Several classes of samples are excluded before conversion, namely SVGs containing gradient definitions, masks, or embedded style blocks. These constructs are not supported by the present representation, which assumes a single solid fill color and a scalar opacity for each shape.

After preprocessing, the SVG document is parsed recursively. Group and root nodes are traversed until individual drawable shapes are reached. For each shape, the fill color is extracted as an RGB triplet and opacity is read from the `opacity` or `fill-opacity` attribute, with percentage values converted to the unit interval. The fill rule is also preserved, because it determines the topological interpretation of nested contours.

Each shape is then rewritten as a path object whose commands can be inspected explicitly. The resulting path is decomposed segment by segment, and every segment is converted to cubic Bezier form. This conversion is exact for native cubic Bezier segments. Straight lines and closing commands use the line-as-cubic special case described above.

Quadratic Bezier segments are elevated to cubic form by the standard degree-elevation formula

$$c_1 = p_0 + \frac{2}{3}(q_1 - p_0), \quad c_2 = p_2 + \frac{2}{3}(q_1 - p_2)$$

where p_0 and p_2 are the original endpoints and q_1 is the quadratic control point. Elliptic arcs are approximated by the parser library `svgelements` [38] as one or more cubic Bezier segments and are stored in the same format.

Consequently, all supported SVG geometry is reduced to a single primitive type.

When the original SVG uses the evenodd fill rule, the converted paths are normalized to the non-zero winding rule used by the internal representation and export. The procedure splits the curve sequence into subpaths, estimates their nesting depth, treats even-depth subpaths as outer boundaries and odd-depth subpaths as holes, and reverses their orientation when necessary. This preserves the filled region while making the shape compatible with the non-zero rule. This conversion is needed because the non-zero rule uses signed contour winding, whereas the even-odd rule uses ray-cross-

ing parity.¹

Once all segments have been converted to cubic Bezier curves and, if needed, their winding order has been normalized, the curve list is partitioned into Bezier paths. A new path is started whenever the start point of the current curve does not coincide with the endpoint of the previous one. Each resulting subpath is stored as one `BezierPath`, and the collection of all subpaths with their common color and opacity forms one `BezierShape`. The final output of the parser is therefore a list of shapes in the same hierarchical form that is subsequently transformed into the fixed-length tensor representation used for training.

4.2.1 Converted Bezier dataset

The conversion procedure described above is used to construct the real raster-to-vector training data for the second stage. This thesis contributes a derived dataset created by converting the SVG files from the source collection described in Section 2.2 into the internal Bezier representation and rasterizing the converted shapes to obtain the conditioning images. The dataset therefore contains paired examples consisting of a fixed-length Bezier tensor and the corresponding raster image.

The converted Bezier dataset is published separately on Hugging Face, with the public artifact listed in Appendix A. Its public split contains 177k training samples, 829 validation samples, and 811 test samples after conversion and filtering, corresponding to approximately 83% of the original default subset. The difference is caused by samples that cannot be represented reliably in the current Bezier format: for example SVGs with gradients, masks, embedded CSS style blocks, patterned fills, or filter effects.

¹The official SVG `fill-rule` definition describes the `nonzero` and `evenodd` algorithms: <https://www.w3.org/TR/2011/REC-SVG11-20110816/painting.html#FillRuleProperty>.

4.3 Synthetic data generator

In addition to SVG data collected from external sources, this work uses a synthetic data generator. Its purpose is to produce a large number of geometrically valid training examples directly in the target Bezier representation. This provides precise control over scene complexity, guarantees compatibility with the representation used by the model, and makes it possible to generate effectively unlimited training data without additional annotation or SVG cleaning.

This property is central to the proposed training strategy. Because the generated vector scene is known exactly, the corresponding raster input can be obtained by rendering, yielding a supervised raster-to-vector pair without any manual labeling. The synthetic generator therefore addresses one of the main data bottlenecks in vector-graphics generation: while captioned SVG datasets are limited and expensive to curate, uncaptioned vector geometry can be synthesized and rasterized cheaply.

The generator produces scenes composed of multiple filled shapes on a square canvas. Each scene contains a random number of objects sampled from a prescribed interval. Every object is represented as one `BezierShape` with one or more closed `BezierPath` contours, a solid RGB color, and an opacity value. Shape opacity is set to one in most cases, while a smaller subset of shapes is drawn with reduced opacity in order to expose the model to moderate transparency variation.

The available shapes are divided into three categories:

- Primitive shapes: circles, ellipses, squares, rectangles, triangles, pentagons, hexagons, and stars. These objects provide simple closed contours with analytically controlled geometry.
- Organic shapes: smooth, rough, and spiky blobs generated from perturbed radial samples. These objects introduce irregular boundaries and more varied local curvature.
- Compound shapes: L-shapes, crosses, arrows, crescents, ring sectors, rounded rectangles, trapezoids, and parallelograms. These objects increase geometric diversity by introducing concavity, varying thickness, and mixed straight and curved boundaries.

This design was chosen to cover both analytically simple contours and shapes with more varied topology and curvature. Examples of generated images are shown in Figure 4.1.



Figure 4.1: Examples of images produced by the synthetic data generator.

All generated geometry is expressed as cubic Bezier curves. Straight polygonal edges are represented by degenerate cubic segments whose control points lie on the corresponding line segment, while circular and elliptical primitives use the cubic arc approximation described in the representation section. Rounded rectangles and related figures combine linear segments with such cubic arc approximations.

Organic blobs are generated differently: first, a set of angles is distributed around a circle with random angular perturbation; second, a radius is sampled independently for each angle; third, the resulting contour points are connected by a closed chain of cubic Bezier segments obtained from a Catmull-Rom-style tangent construction [39]. The handle length is scaled by a smoothness parameter, which allows the generator to control whether the blob is smooth, rough, or spiky.

For each sampled shape, geometric parameters such as size, aspect ratio, rotation, and contour detail are drawn from random intervals that depend on the shape category. The object center is then sampled under a margin constraint so that the entire shape remains inside the canvas with high probability. For this purpose, the generator estimates how far the transformed shape can extend from its center and samples only center positions that leave enough margin to the canvas boundary. After construction, the outer contour is explicitly oriented counter-clockwise. This establishes a consistent winding convention for all synthesized objects.

Some shapes may additionally contain holes. Hole generation is only attempted for shape types for which an internal contour can be inserted reliably. Candidate hole contours are sampled inside the bounding box of the outer shape, scaled to a random fraction of its size, and randomly offset away from the exact center. The hole itself may again be circular, polygonal, blob-like, or rounded-rectangular. In contrast to the outer boundary, the hole contour is forced to clockwise orientation. This opposite winding is necessary because the exported SVGs use the non-zero fill rule, under which opposite contour directions produce cut-out regions.

A complete synthetic scene is constructed by repeatedly sampling shapes until either the requested number of shapes is reached or the global segment budget is exhausted. The segment budget is important because the downstream model expects a fixed maximum number of Bezier segments per sample. Instead of generating a scene first and truncating it afterwards, the generator stops adding shapes as soon as the next object would exceed the allowed number of segments. This ensures that every produced scene is valid without altering already generated geometry.

The synthetic dataset interface generates samples on the fly during training. The stream of synthetic scenes is effectively unbounded,

so pretraining is not limited to a fixed corpus of generated files. Each generated scene is converted to the internal tensor representation, serialized back to SVG, rasterized to an RGB image, and finally processed by the image encoder preprocessing pipeline. The dataset therefore returns the same type of data as the real dataset, namely a tensor of Bezier segments and a corresponding conditioning raster image. The same training and sampling code can therefore consume synthetic and real data.

4.4 Model architecture

The architecture must solve two coupled problems. It must generate a continuous Bezier tensor, and it must keep that tensor aligned with the raster image used as conditioning. A conditional flow-matching model is suitable for the first requirement because it learns a time-dependent velocity field in the same continuous representation space as the Bezier descriptors [16]. A transformer backbone is suitable for the second requirement because it can model dependencies among segment tokens while attending to visual features from the input image [40].

The predictive model is therefore a conditional flow-matching transformer, shown at a high level in Figure 4.2. Its input consists of two parts: a sequence of noisy Bezier-segment descriptors and a raster conditioning image. The output is a sequence of the same length and dimensionality as the Bezier input, interpreted as a velocity field in representation space. The architecture operates directly on the continuous tensor representation introduced above and predicts how a noisy sample should move toward a valid vector graphic conditioned on the raster image.

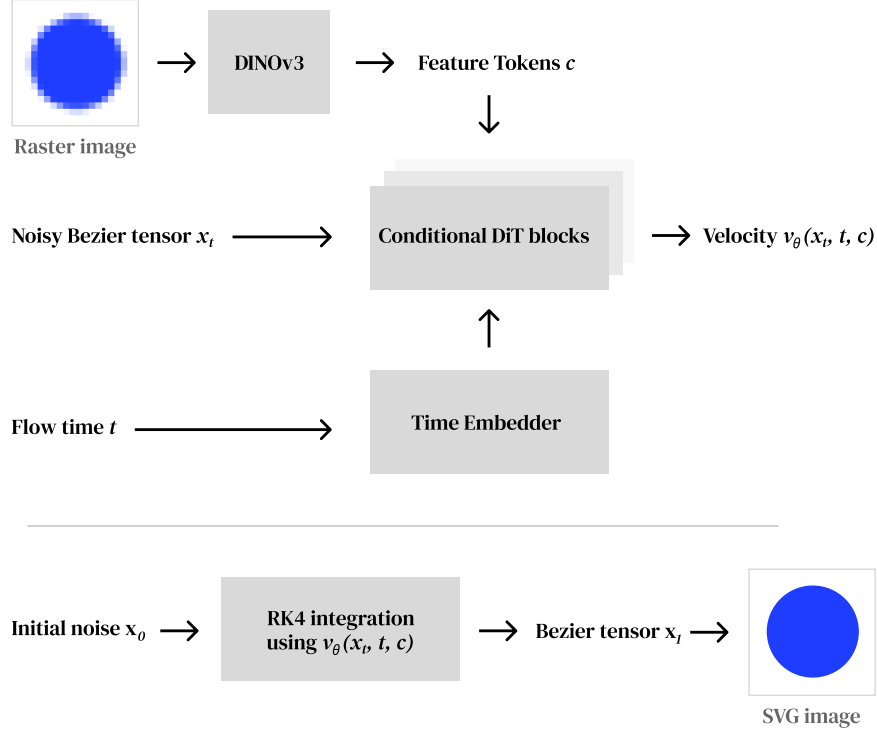


Figure 4.2: High-level structure of the proposed conditional flow-matching vectorizer.

4.4.1 Training objective

Training follows the rectified-flow formulation [41]. Let x_1 denote a ground truth Bezier tensor sampled from the dataset and let x_0 be Gaussian noise of the same shape. A scalar time t is sampled for each training example from a logit-normal distribution obtained by applying the sigmoid function to a standard normal sample, following timestep sampling used in rectified-flow transformer training [42].

The noisy intermediate point is then constructed by linear interpolation

$$x_t = tx_1 + (1 - t)x_0$$

The target velocity is defined as

$$v^* = x_1 - x_0$$

Given x_t , t , and the image-conditioning tokens, the network predicts a velocity field $v_{\theta(x_t, t, c)}$ and is optimized using the mean squared error objective

$$L = \|v_{\theta(x_t, t, c)} - v^*\|_2^2$$

In the current implementation, this loss is evaluated over the full sequence, including padded positions.

The model uses conditioning dropout during training, as recommended by [29] to enable classifier-free guidance and improve generalization. With a fixed probability, the image-conditioning sequence is replaced by a learned null token broadcast across the conditioning length. This teaches the network both conditional and unconditional velocity fields within a single set of parameters. Preliminary experiments also showed that conditioning dropout improved generalization to validation inputs, so it was retained as part of the final training setup. During inference, the two predictions can be combined as

$$v = v_u + w(v_c - v_u)$$

where w is the guidance scale. When $w = 1$, standard conditional sampling is recovered.

4.4.2 Network architecture

The conditioning image must provide more than raw pixel values: the vectorizer needs cues about object boundaries, part structure, and broader visual semantics. For this reason, the conditioning branch uses a pretrained DINOv3 visual encoder [43], specifically facebook/dinov3-vits16-pretrain-lvd1689m. DINOv3 is a self-supervised visual foundation model designed to produce transferable visual features across a broad range of downstream tasks [43]. In this work, the encoder is kept frozen throughout training and is used only

to extract a sequence of visual features from the conditioning raster image. Freezing the image encoder reduces the number of trainable parameters and stabilizes optimization, while still providing semantically rich image descriptors.

Concretely, the model takes the last hidden state of DINOv3 and linearly projects it to the internal hidden dimension of the transformer. This yields a sequence of conditioning tokens that serve as keys and values in cross-attention.

The Bezier branch processes a tensor of segment descriptors of shape (B, N, D) , where B is batch size, N is the maximum number of segments, and $D = 13$ is the segment dimensionality, covering coordinates, color, opacity, path-structure flags, and the validity flag.

Each segment vector is projected by a learned linear layer into a hidden space of dimension H . The interpolation time $t \in [0, 1]$ from the rectified-flow objective is embedded separately using sinusoidal features followed by a multilayer perceptron [40]. The resulting time embedding is then used to modulate all transformer blocks through adaptive layer normalization.

The backbone itself is a stack of transformer blocks of DiT type [4]. Each block contains three sublayers:

- RoPE self-attention over the Bezier token sequence.
- Cross-attention from Bezier tokens to image-conditioning tokens.
- A position-wise feed-forward network.

Self-attention uses rotary positional embeddings applied to the query and key vectors [44]. This gives the Bezier-token stream information about the order of segments within the sequence while preserving the attention-based formulation. Cross-attention does not add separate rotary embeddings; instead, it uses the position-aware Bezier hidden states as queries and the DINOv3 patch features, which already contain spatial information from the visual encoder, as keys and values. In this way, each Bezier token can attend to relevant visual features while combining geometric context from the partially denoised vector sequence with semantic and structural cues present in the conditioning image.

Each transformer block is modulated by the time embedding using adaptive layer normalization with gating. More precisely, the time embedding is passed through a small modulation network that predicts, for each of the three sublayers, a shift vector, a scale vector,

and a residual gate. If x denotes a normalized token representation, the modulation takes the form

$$\text{mod}(x) = x \cdot (1 + \gamma) + \beta$$

where β and γ are functions of the time embedding.

The gated residual connection then controls how strongly the output of the corresponding sublayer is injected back into the main stream. This adaptive normalization follows the conditioning mechanism used in DiT, where timestep and class-conditioning information is used to modulate transformer residual blocks [4].

In this thesis, the same mechanism is used for the flow time: it allows the network to vary its computation with t , matching the flow-matching formulation in which the learned velocity is a time-dependent vector field [16].

After the stacked transformer blocks, the model applies one final adaptive normalization step conditioned on time and then projects the hidden representation back to the original Bezier-segment dimension. The final linear projection is initialized with zeros, so the network initially predicts a near zero velocity field. This follows the zero-initialization principle used in DiT-style adaptive layer normalization, where residual branches are initialized to make the transformer blocks behave close to the identity at the beginning of training [4]. In the present flow-matching setting, this avoids large uncontrolled updates before the model has learned a meaningful time-dependent vector field.

Sampling is performed by solving the learned ordinary differential equation from noise toward data. The process starts from an initial sample

$$x(0) \sim N(0, I)$$

The model then integrates the velocity field from $t = 0$ to $t = 1$ using the classical fourth-order Runge-Kutta method [45] with a fixed number of time steps. In each integration step, the transformer is evaluated one or more times to obtain the required intermediate velocities. The final state is interpreted as a predicted Bezier tensor,

which is subsequently converted back to vector shapes and rendered as SVG. This sampling procedure is deterministic for fixed initial noise, fixed conditioning, and fixed integration parameters.

4.5 Training schedule

Synthetic data and real SVG data provide complementary advantages for the vectorizer. Synthetic data offer unlimited quantity and precise control over geometric variation, while real SVG data provide more realistic structure, stylistic diversity, and distributional properties closer to the target use case. The training schedule therefore uses two consecutive phases: pretraining on data generated procedurally in the Bezier representation, followed by fine-tuning on the SVG dataset derived from real vector graphics.

This distinction is important because automatic vectorization is underdetermined from pixels alone: when the vector scene is generated procedurally, the exact geometric target is known by construction, whereas for ordinary raster images there may be many plausible vector explanations [9,10].

4.5.1 Synthetic pretraining

The first phase is intended to teach the model the basic grammar of vector graphics before it has to model the greater variability of real SVG files. This includes curve continuity, path organization, contour winding, color consistency within shapes, and the general relationship between raster appearance and vector structure. To provide this signal, the model is exposed to procedurally generated scenes containing simple primitives, compound shapes, blobs, and shapes with holes. Because these data are generated directly in the target Bezier representation, they are guaranteed to be geometrically valid and structurally consistent.

The effectively unlimited size of the synthetic dataset also reduces the risk of overfitting and allows controlled experiments with scene complexity, segment count, and object diversity.

In the current experimental setup, synthetic pretraining was performed on a single NVIDIA H200 GPU with batch size 256 for approximately 10 days, totaling about 1.5 million optimization steps. The model used hidden size 768, 16 transformer layers, 12 attention heads, and a maximum sequence length of 256 Bezier segments. Optimization used AdamW with learning rate $1e - 4$, bfloat16 mixed precision, gradient clipping with norm 1.0, and FlashAttention 2 [46]. Conditioning dropout was applied with probability 10%. Synthetic scenes contained between 1 and 10 shapes, and the checkpoint was selected according to the lowest validation rendered-image MSE.

4.5.2 Fine-tuning on the SVG dataset

Pretraining alone cannot expose the model to the full variability of real vector graphics. Compared with the synthetic generator, real SVG data contain richer compositions, more varied contour structures, and a broader range of design conventions. The model is therefore fine-tuned on SVG files converted into the internal Bezier representation, which adapts it from the simplified synthetic distribution to the final task distribution.

Fine-tuning was initialized from the selected synthetic-pretraining checkpoint. The run was performed on a single NVIDIA L40S GPU for approximately 6 days, covering about 1.5 million optimization steps over 600 epochs. The checkpoint used for evaluation was again selected according to the lowest validation rendered-image MSE. Optimization used AdamW with learning rate $5e - 5$, 1000 warmup steps, bfloat16 mixed precision, gradient clipping with norm 1.0, and conditioning dropout with probability 10%.

4.6 Evaluation

The Stage 2 experiments evaluate the conditional flow-matching vectorizer. First, architectural ablations compare the flow-matching formulation with an autoregressive variant of comparable size, and measure the effect of using a pretrained image encoder for the raster input. The image-encoder ablation compares the full model with a

model trained on raw raster inputs while keeping the vectorizer backbone as constant as possible. Together, these experiments clarify how much of the performance is due to the flow-matching objective, the transformer backbone, and the pretrained visual representation.

4.6.1 Architecture ablations

To separate the effect of the generative formulation from the effect of model capacity, the flow-matching vectorizer was also compared with an autoregressive variant. The autoregressive model uses the same hidden size, number of layers, maximum sequence length, and DINOv3 image encoder, but predicts the Bezier sequence step by step rather than learning a continuous denoising vector field. The comparison in Figure 4.3 is capped at the first 250k training steps, where both runs have logged train and validation image-space MSE.

The autoregressive model reduces the reconstruction error during training, but remains consistently worse than the flow-matching model on both splits. This suggests that, for this fixed Bezier representation and model scale, the flow-matching objective provides a more effective training signal than next-step autoregressive prediction.

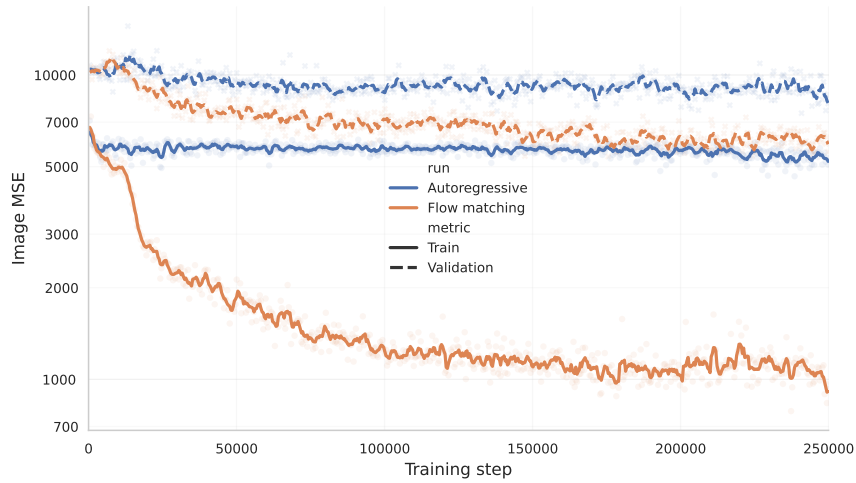


Figure 4.3: Flow-matching and autoregressive vectorizer comparison.

The image representation was further evaluated by comparing the full architecture with a variant trained without the image encoder. In the ablated model, the vectorizer is still conditioned on the raster input, but receives it in a raw form rather than through DINOv3 feature tokens. This comparison therefore tests whether the pretrained visual representation provides more useful input-specific guidance than directly exposing the vectorizer to the image data.

The results in Figure 4.4 and Figure 4.5 are limited to the first 150k training steps. The model with the image encoder reaches lower train and validation reconstruction MSE and also maintains a lower training loss over the shared interval. The difference is especially important on the validation split, where the encoder-based model can adapt the generated Bezier curves to the observed raster image more effectively than the raw-input variant. This supports the use of a pretrained image encoder as a central part of the conditional vectorizer.

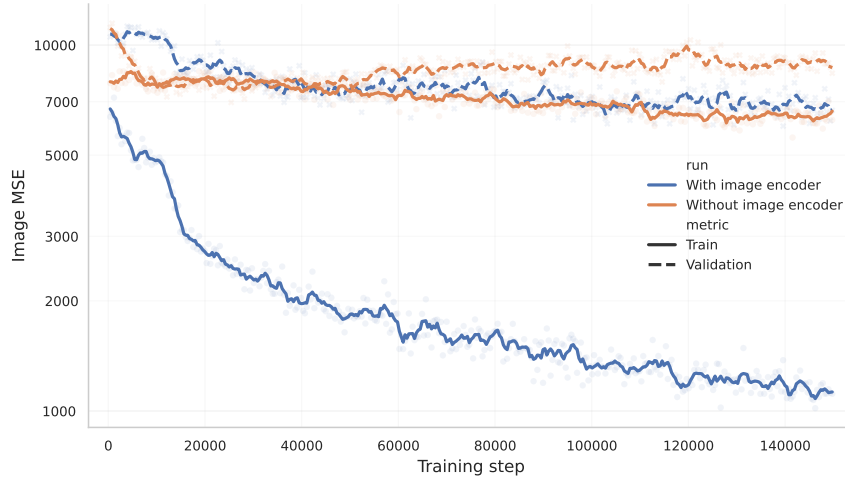


Figure 4.4: Image reconstruction error with encoded and raw raster input.

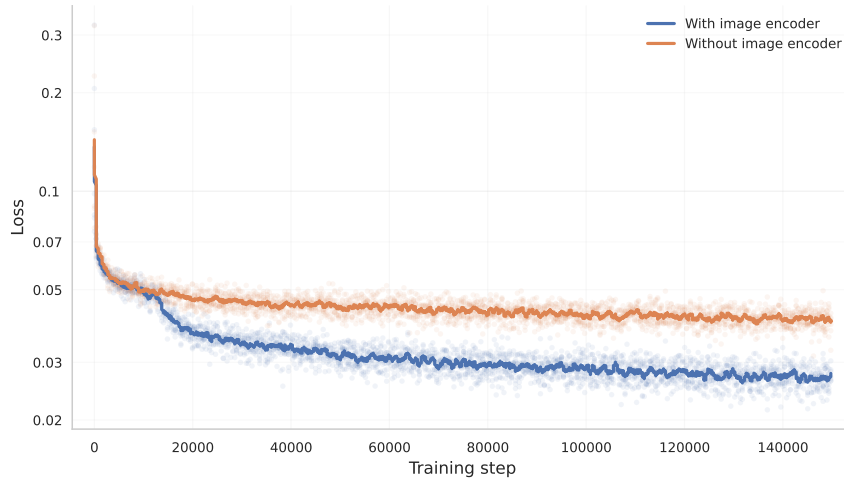


Figure 4.5: Training loss with encoded and raw raster input.

The selected architecture is then used for synthetic pretraining before fine-tuning on the SVG Repo data. This training setup tests the central hypothesis that synthetic Bezier data provide a useful geometric prior even though they are simpler than real SVG graphics.

4.6.2 Synthetic pretraining results

The optimization dynamics of this pretraining run are summarized in Figure 4.6 and Figure 4.7. The training objective decreases rapidly during the initial phase and then enters a slower refinement regime, indicating that the model first learns coarse Bezier-structure prediction before improving smaller geometric and appearance errors. The image-space MSE is measured by rendering predicted vectors back to raster images and comparing them with the corresponding targets. In Figure 4.7, the training curve is evaluated on synthetic pretraining samples, whereas the validation curve is evaluated on validation samples from the SVG dataset. The metric therefore provides a complementary reconstruction-oriented view of both pretraining quality and transfer to real SVG-derived data, in addition to the direct flow-matching loss.

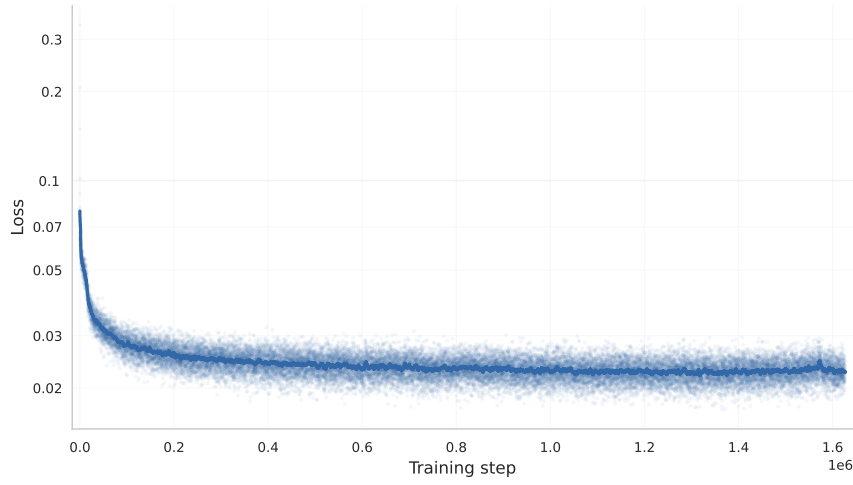


Figure 4.6: Synthetic pretraining loss of the raster-to-vector model.

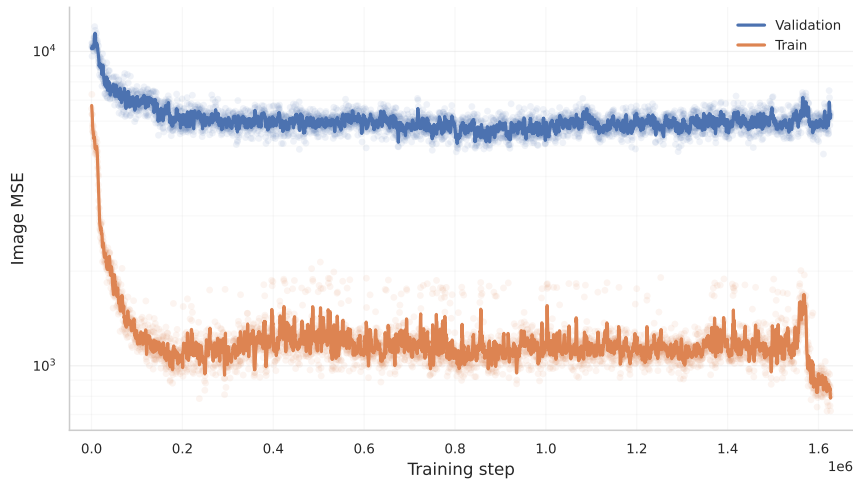


Figure 4.7: Image reconstruction error during synthetic pretraining.

Qualitative samples from the final synthetic pretraining checkpoint are shown in Table 4.1. The training examples indicate that the model has learned to vectorize samples from the synthetic generator: the predicted Bezier representations preserve the main silhouettes, colors, and compound-shape structure of the references. The validation examples further suggest that this learned geometric prior transfers to data from the SVG dataset, but they also expose a remaining domain gap caused by the visual and structural differences between

procedurally generated scenes and real vector graphics. This gap motivates the subsequent fine-tuning stage on real SVG-derived data. In each case, the generated image is produced by sampling the flow-matching vectorizer conditioned on the corresponding raster reference and then rendering the predicted Bezier representation back to an image.

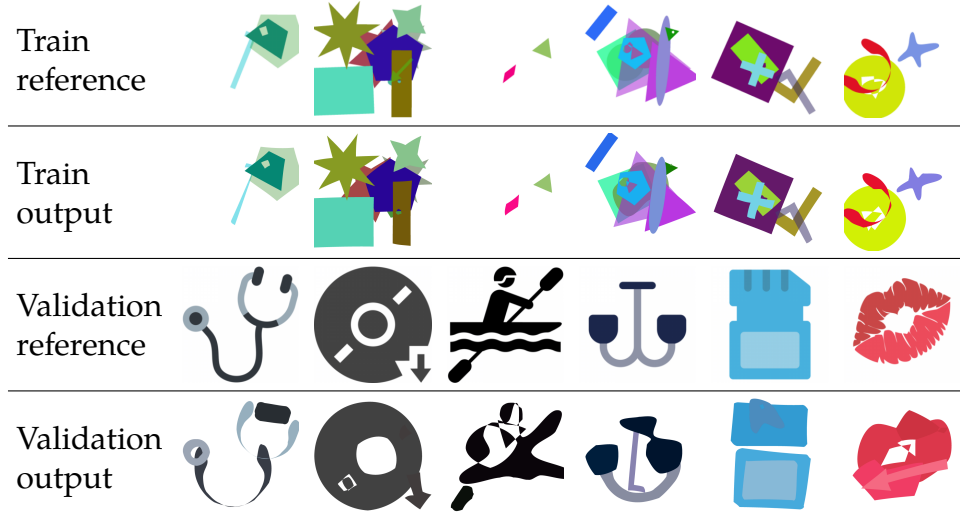


Table 4.1: Qualitative samples from the synthetic pretraining checkpoint.

4.6.3 Fine-tuning results

After synthetic pretraining, the same raster-to-vector model is fine-tuned on the SVG Repo dataset. The fine-tuning dynamics are shown in Figure 4.8 and Figure 4.9. The training loss starts substantially lower than in the synthetic pretraining run, which is consistent with initialization from the pretrained checkpoint, and continues to decrease during adaptation to real vector graphics. The image-space MSE remains noisier than the direct training objective, but it tracks the rendered reconstruction quality on both training and validation samples and therefore complements the loss curve.

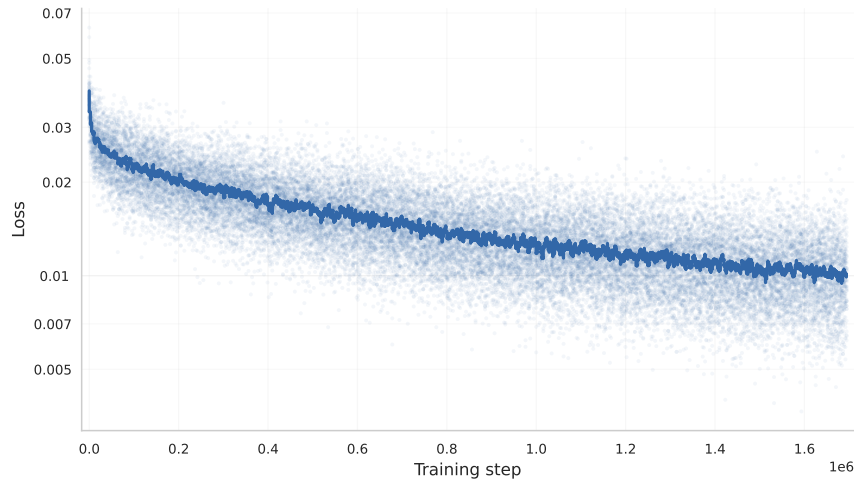


Figure 4.8: Fine-tuning loss of the raster-to-vector model on SVG Repo data.

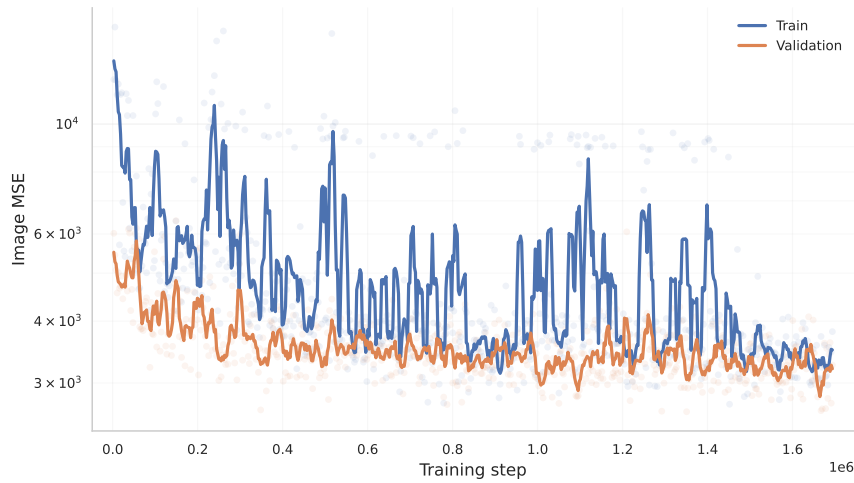


Figure 4.9: Image reconstruction error during fine-tuning on SVG Repo data.

4.6.4 Vectorization benchmark on controlled inputs

Before evaluating the full text-to-vector pipeline, the vectorization stage is evaluated in isolation. This benchmark uses clean raster inputs obtained either from the SVG validation split or from the

synthetic generator. These inputs are not produced by the text-to-raster model; they are controlled sources with known reference SVGs or known procedural structure. The purpose of this part is therefore to measure raster-to-vector quality under reproducible conditions, rather than to claim end-to-end pipeline performance.

The comparison includes classical vectorization tools and recent neural SVG-vectorization systems. It emphasizes not only pixel-level reconstruction, but also properties important for editable vector graphics, such as path count, node count, topological cleanliness, robustness to controlled input variation, and ease of manual editing.

The vectorization comparison is performed with an evaluation script that renders each reference SVG and each generated SVG at a fixed resolution of 1024 pixels, compares the rendered RGB images, and records both image-space and structure-related statistics. The comparison contains four methods: the proposed flow-matching vectorizer, OmniSVG [8], StarVector [7], and vtracer [19]. All methods are evaluated on the same reference set and with the same rasterization settings.

Qualitative behavior

The quantitative comparison is paired with two qualitative grids. The first grid uses samples from the SVG validation split. These examples are useful for checking performance on the target distribution, but they should be interpreted as in-distribution examples: the proposed model is trained on the same dataset family, and large external SVG models may also have been exposed to visually similar icon data during pretraining. The validation grid therefore shows how well the methods handle the type of data used in the main benchmark, rather than proving broad vectorization ability. The validation examples are shown in Table 4.2.

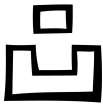
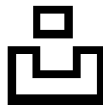
Reference						
Ours 0.26B						
OmniSVG1.1 4B						
OmniSVG1.1 8B						
StarVector 1B						
StarVector 8B						

Table 4.2: Qualitative comparison on SVG validation samples. Empty cells indicate invalid output.

The second qualitative grid uses samples from the synthetic generator. In this setting, the reference vector structure is produced by a controlled procedural process rather than collected from the same icon distribution as the validation set. This makes the comparison a more direct test of general raster-to-vector capability: the methods must recover clean geometric structure from rendered images whose underlying shapes, holes, intersections, and curve configurations are known. The examples are selected by fixed criteria such as sample index and input source, which avoids choosing only visually favorable cases. The synthetic examples are shown in Table 4.3.

Reference						
Ours 0.26B						
OmniSVG1.1 4B						
OmniSVG1.1 8B						
StarVector 1B						
StarVector 8B						

Table 4.3: Qualitative comparison on synthetic-generator samples. Empty cells indicate invalid output.

Rendered fidelity

The quantitative metrics are computed after rendering both SVGs to RGB images at the same resolution. MSE is the mean squared difference between corresponding RGB pixel values, reported on the 0–255 intensity scale. It measures direct raster reconstruction error and is sensitive to both color differences and small spatial misalignments. SSIM compares the rendered images using luminance, contrast, and covariance statistics on normalized RGB values; it is included because two vectorizations can have similar pixel error while preserving global structure to different degrees [47]. In this implementation SSIM is computed globally per channel and then averaged, so it should be interpreted as a coarse structural score rather than as a full windowed perceptual metric.

The remaining fidelity metrics emphasize foreground shape and boundary alignment. Mask IoU thresholds each rendered image

into foreground and background, treating pixels darker than the foreground threshold as foreground, and measures the intersection-over-union of the two masks, corresponding to the Jaccard similarity coefficient [48]. Boundary F1 first extracts edge points from the rendered images and then measures precision and recall under a fixed pixel tolerance, following the common boundary-evaluation idea of matching contours within a localization tolerance [49].

The tables report the 2 px tolerance, which rewards methods whose contours are close to the reference even when the filled regions are not identical. Chamfer distance averages the nearest-edge distance in both directions between the reference and generated contours [50], whereas Hausdorff distance reports the worst nearest-edge discrepancy [51]. Chamfer therefore measures typical contour alignment, while Hausdorff is more sensitive to outliers such as missing strokes, distant artifacts, or a single badly placed shape. The rendered fidelity results are reported in Table 4.4 and Table 4.5.

Method	MSE (0-255) ↓	SSIM ↑	Mask IoU ↑	Boundary F1 at 2 px ↑	Chamfer px ↓	Hausdorff px ↓
Ours 0.26B	7107.52	0.653	0.644	0.324	16.04	103.24
OmniSVG1.1 4B	7696.39	0.621	0.631	0.538	17.84	145.46
OmniSVG1.1 8B	8425.56	0.589	0.608	0.516	18.89	149.69
StarVector 1B	5147.82	0.652	0.631	0.483	24.26	143.28
StarVector 8B	8449.48	0.461	0.444	0.441	38.75	229.73
vtracer	92.01	0.994	0.984	0.886	1.26	17.20

Table 4.4: Vectorization fidelity on SVG validation samples.

4. STAGE 2: VECTORIZATION

Method	MSE (0-255) ↓	SSIM ↑	Mask IoU ↑	Boundary F1 at 2 px ↑	Chamfer px ↓	Hausdorff px ↓
Ours 0.26B	2432.57	0.725	0.758	0.390	14.50	108.82
OmniSVG1.1 4B	9591.49	0.330	0.432	0.461	32.87	242.00
OmniSVG1.1 8B	11024.09	0.283	0.407	0.436	36.07	253.81
StarVector 1B	6339.14	0.314	0.297	0.476	47.02	317.25
StarVector 8B	7536.35	0.137	0.104	0.496	59.58	401.38
vtracer	23.95	0.997	0.993	0.920	1.03	20.45

Table 4.5: Vectorization fidelity on synthetic-generator samples.**SVG validity and editability**

SVG validity and complexity are reported separately from visual fidelity. The valid SVG rate is the fraction of generated files that can be rendered without error; missing or non-renderable files are replaced by a white image for fidelity scoring but are counted as failures in the validity table. SVG bytes, element count, path count, and path-command count are simple proxies for output complexity. Lower values indicate a more compact and potentially more editable SVG only when the corresponding fidelity metrics remain competitive, because a trivially simple file can also be inaccurate. The validity and complexity results are summarized in Table 4.6 and Table 4.7.

4. STAGE 2: VECTORIZATION

Method	Valid SVG rate ↑	SVG bytes ↓	Elements ↓	Paths ↓	Path commands ↓
Ours 0.26B	100.0%	10329.02	5.27	4.27	102.55
OmniSVG1.1 4B	99.4%	5284.03	5.04	4.04	219.53
OmniSVG1.1 8B	99.3%	5296.17	8.62	7.62	206.38
StarVector 1B	79.0%	1957.71	9.17	4.09	118.60
StarVector 8B	65.1%	2220.83	10.63	5.36	213.25
vtracer	100.0%	14370.11	10.68	9.68	364.55

Table 4.6: SVG validity and complexity on SVG validation samples.

Method	Valid SVG rate ↑	SVG bytes ↓	Elements ↓	Paths ↓	Path commands ↓
Ours 0.26B	100.0%	9090.44	9.66	8.66	90.17
OmniSVG1.1 4B	99.2%	9165.78	13.72	12.72	393.75
OmniSVG1.1 8B	98.6%	9658.08	29.95	28.95	400.92
StarVector 1B	43.2%	4108.50	30.42	9.97	447.18
StarVector 8B	20.5%	5354.53	40.59	14.22	962.31
vtracer	100.0%	24468.73	14.66	13.66	625.33

Table 4.7: SVG validity and complexity on synthetic-generator samples.**Summary**

The validation split shows the expected behavior of the proposed vectorizer. It preserves global structure and foreground regions competitively among the neural methods, but it is weaker on fine boundary agreement. This follows from the fixed-capacity Bezier representation: the model must approximate the raster with a limited number of segments instead of tracing every edge.

The synthetic-generator split tests the same model outside the SVG Repo validation distribution. In this setting, the proposed

method leads the neural methods on MSE, SSIM, Mask IoU, Chamfer distance, and Hausdorff distance, while StarVector has higher Boundary F1. It also produces valid SVGs for every sample. The autoregressive baselines lose reliability on this split, especially the StarVector models, which makes validity part of the result rather than an implementation detail.

vtracer remains the best method when the goal is raster reconstruction only. Its cost is structural complexity: it uses far more paths and path commands than the neural methods. The proposed model trades some visual accuracy for a simpler SVG while keeping a 100% valid-output rate on both splits. This result is obtained with a 0.26B-parameter model, about 4 times smaller than StarVector 1B, about 15 times smaller than OmniSVG 4B, and about 31 times smaller than the 8B baselines.

5 End-to-End Pipeline Evaluation

The previous vectorization chapter uses either rasterized SVG validation samples or controlled procedural images. A separate evaluation is needed for the actual output of the text-to-raster stage, because generated raster images have a different error profile from both sources. This chapter is the pipeline evaluation: unlike the controlled benchmark in the previous chapter, it measures vectorization of images rendered by the text-to-raster generator rather than vectorization of clean dataset or synthetic inputs. The reference rasters in this evaluation are produced by the Z-Image Base prefixed + LoRA configuration selected in Section 3.3; the turbo-prefixed LoRA variant discussed there is retained as the faster alternative.

The Z-Image stage may produce anti-aliased contours, slight texture, local color variation, incomplete boundaries, or other artifacts that are not present in the synthetic generator, while also differing from clean SVG Repo renderings. This setting therefore measures the practical interface between the text-to-raster model and a raster-to-vector converter.

Unlike the controlled vectorization benchmark script, which compares a generated SVG against a reference SVG after rendering both files, this experiment uses a separate evaluation script that compares a generated PNG directly against the SVG obtained from that PNG. Each input raster is rendered from the Z-Image pipeline, vectorized with `vtracer`, rendered back to a PNG at the same resolution, and compared with the original raster image.

The reported numbers therefore evaluate raster-vector-raster consistency on generated images, not semantic agreement with a ground-truth SVG. Qualitative inspection should focus on whether the vectorization preserves the main shape layout, fill regions, and contour smoothness, and whether small raster artifacts are converted into unnecessary paths. Quantitative scores summarize the same behavior over the full generated set. Qualitative examples are shown in Table 5.1, rendered-fidelity results in Table 5.2, and validity and complexity results in Table 5.3.

Reference						
Ours 0.26B						
OmniSVG1.1 4B						
OmniSVG1.1 8B						
StarVector 1B						
StarVector 8B						
vtracer						

Table 5.1: Qualitative comparison of vectorizers on rasters generated by the selected Stage 1 model. The reference row contains the input rasters from the Z-Image Base prefixed + LoRA configuration from Section 3.3. Empty cells indicate invalid output.

5. END-TO-END PIPELINE EVALUATION

Method	MSE (0-255) ↓	SSIM ↑	Mask IoU ↑	Boundary F1 at 2 px ↑	Chamfer px ↓	Hausdorff px ↓
Ours 0.26B	6004.36	0.681	0.651	0.371	15.62	102.79
OmniSVG1.1 4B	8973.13	0.550	0.549	0.527	22.15	167.65
OmniSVG1.1 8B	9923.60	0.526	0.532	0.503	24.05	172.41
StarVector 1B	5057.47	0.729	0.690	0.417	13.09	88.63
StarVector 8B	6592.78	0.622	0.596	0.427	18.50	120.81
vtracer	145.45	0.984	0.971	0.866	1.93	27.91

Table 5.2: Vectorization fidelity on generated raster images.

Method	Valid SVG rate ↑	SVG bytes ↓	Elements ↓	Paths ↓	Path commands ↓
Ours 0.26B	100.0%	8005.76	4.67	3.67	79.53
OmniSVG1.1 4B	99.2%	6696.20	6.46	5.46	244.43
OmniSVG1.1 8B	98.6%	6803.54	10.89	9.89	250.28
StarVector 1B	49.0%	2102.44	11.49	2.87	40.14
StarVector 8B	50.5%	1270.68	7.10	2.31	44.36
vtracer	100.0%	60192.09	95.84	94.84	1779.18

Table 5.3: SVG validity and complexity on generated raster images.

On generated rasters, the proposed vectorizer is not the best raster reconstructor. vtracer preserves the bitmap most accurately, and StarVector 1B is stronger on several fidelity metrics among neural methods. The proposed model instead gives the most reliable neural output: all generated inputs produce renderable SVGs, with substantially fewer paths and commands than classical tracing.

Generated rasters contain anti-aliasing, imperfect boundaries, local texture, and other artifacts from the first stage. vtracer follows these details and therefore produces large SVGs. The proposed model suppresses some fine detail, which hurts contour precision, but keeps the result closer to a compact object-level drawing.

The autoregressive baselines show the remaining trade-off. Some successful outputs are more faithful or more compact than the proposed model, but many outputs are invalid in this setting. That failure is critical in an end-to-end pipeline because an invalid second-stage SVG breaks the generated result. The 0.26B flow-matching model avoids this failure while being about 4 times smaller than StarVector 1B, about 15 times smaller than OmniSVG 4B, and about 31 times smaller than the 8B baselines.

5.1 End-to-end text-to-SVG comparison

The quantitative evaluation above measures the behavior of the vectorization stage on generated rasters, but the full goal of the thesis is a text-to-SVG system. It is therefore useful to also inspect complete outputs where a text prompt is first rendered by the adapted Z-Image stage and then converted into SVG by a vectorization stage.

Table 5.4 compares the proposed two-stage pipeline with OmniSVG, which generates SVG directly from text. The pipeline variant shown in the figure uses the adapted Z-Image Base prefixed + LoRA model followed by vtracer, because this combination gives the strongest rendered fidelity in the end-to-end experiments. This comparison therefore isolates the practical value of the pipeline decomposition: text understanding and visual composition are handled by a pretrained raster generator, while the final SVG is obtained by a separate vectorization step.

StarVector is omitted from this comparison because its public 1B and 8B checkpoints are released for image-to-SVG vectorization, not text-to-SVG generation [52].

The examples show a clear qualitative advantage for the pipeline formulation on these prompts. The proposed pipeline usually preserves the requested object and icon-like composition, whereas both OmniSVG variants often produce incomplete, distorted, or semantically mismatched drawings. The direct text-to-SVG models remain attractive because they avoid an intermediate raster image, but these examples indicate that the intermediate raster representation can make the overall system substantially more robust.

5. END-TO-END PIPELINE EVALUATION

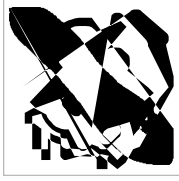
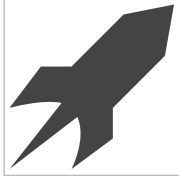
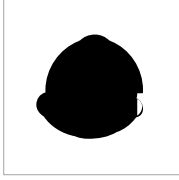
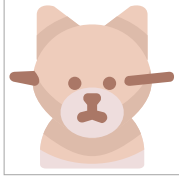
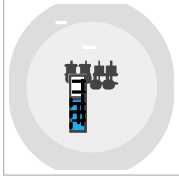
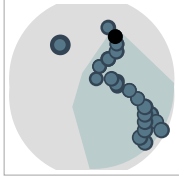
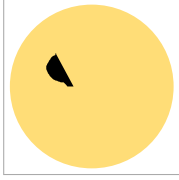
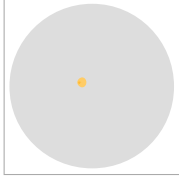
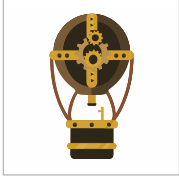
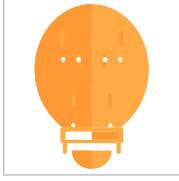
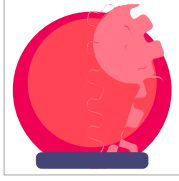
Prompt	Proposed pipeline Z-Image + vtracer	Direct text-to-SVG OmniSVG1.1 4B	Direct text-to-SVG OmniSVG1.1 8B
a simple lighthouse icon			
A minimal rocket icon			
A cute cat face icon			
A magic potion bottle with bubbles, sparkles, cork, and label			
A cozy cabin in the woods with a chimney, pine trees, and a crescent moon			
A steampunk hot air balloon with gears, ropes, basket, and brass details			

Table 5.4: End-to-end text-to-SVG examples generated by the proposed two-stage pipeline and by direct OmniSVG baselines.

Conclusion

This thesis delivers *a generative-AI pipeline for producing SVG graphics from text prompts*. The pipeline is organized into two stages: a semantic stage that generates a raster image matching the prompt, and a geometric stage that converts the raster image into *a valid, compact, editable SVG*. This decomposition makes the individual stages easier to train and evaluate while still supporting end-to-end SVG generation from unseen prompts. In the qualitative end-to-end comparison, the proposed pipeline clearly outperforms the tested direct OmniSVG baselines: it preserves the requested objects and icon-like compositions, while the baselines frequently produce incomplete, distorted, or semantically mismatched SVGs.

The best rendered-image fidelity in the end-to-end experiments is achieved by *the image-generation model adapted in this thesis from Z-Image*, followed by *vttracer*. This combination preserves the generated raster well, but its SVG outputs are large and contain many paths and commands. Such files render correctly, but they are much less useful as editable vector graphics.

The main methodological contribution of the second stage is a novel formulation of neural SVG vectorization. Publicly available neural baselines considered in this thesis generate SVGs autoregressively, as token or command sequences. This thesis instead formulates vectorization as conditional flow matching over a fixed-size Bezier representation. The resulting vectorizer does not yet match *vttracer* in pixel-level reconstruction, but it is the most reliable neural vectorizer in the tested setting: despite being 4–31 times smaller than the autoregressive baselines, it produces valid SVGs consistently, keeps the output compact, and avoids their severe validity failures. It is not a replacement for classical tracing when visual fidelity is the only goal; it is a step toward SVGs generated as structured objects rather than dense traces of pixels.

The work also demonstrates supervised pretraining of an SVG vectorizer on procedurally generated vector data. The synthetic generator provides effectively unlimited raster-vector pairs with exact labels, allowing the model to first learn a geometric raster-to-Bezier prior and then adapt to real SVG collections during fine-tuning.

The implemented pipeline shows that text-to-SVG systems should be evaluated not only by rendered-image similarity, but also by *SVG validity, compactness, and editability*. Classical tracing currently wins on visual reconstruction. The proposed neural vectorizer has lower rendered fidelity than tracing, but better compactness and stronger validity than the tested neural baselines. Scaling this approach with more data and larger models may improve visual fidelity while preserving the validity and compactness advantages observed in the experiments.

The public implementation and trained model artifacts are listed in [Appendix A](#).

5.2 Limitations

Several limitations remain. The current vector representation expresses all geometry as cubic Bezier segments, so higher-level SVG primitives such as circles, rectangles, and symbolic shape elements are not preserved as separate objects. Although these primitives can be approximated accurately by Bezier curves, the resulting SVG is less semantically editable than an SVG that keeps the original primitive types. The data pipeline also assumes solid fills with opacity and does not parse gradients, masks, filters, or complex style rules. This excludes a significant part of the SVG design space. In addition, the fixed maximum number of segments imposes a hard capacity limit: graphics requiring more segments must either be simplified or truncated.

5.3 Future work

Future work should focus on improving both the expressiveness of the representation and the reliability of the full pipeline.

One direction is primitive recovery: closed contours could be tested for compatibility with circles, ellipses, rectangles, rounded rectangles, polygons, or simple compound shapes, and then replaced

by the corresponding SVG elements when the approximation error is sufficiently small.

Another direction is to extend the data conversion and renderer to gradients, transparency, masks, and richer style attributes. The current pipeline assumes a white background in both the text-to-raster stage and the vectorizer training setup. Producing and training on transparent raster inputs would better match the common use of SVG graphics as foreground assets; methods for transparent image generation, such as latent-transparency diffusion [53], suggest one possible way to adapt the first stage.

Finally, larger paired vector datasets and stronger conditioning could make it possible to revisit direct text-to-vector generation, but the results of this thesis indicate that the two-stage formulation remains a useful and data-efficient approach for further research.

Bibliography

1. WORLD WIDE WEB CONSORTIUM. *Scalable Vector Graphics (SVG) 1.1 (Second Edition)*. 2011. URL: <https://www.w3.org/TR/2011/REC-SVG11-20110816/>.
2. HO, Jonathan, JAIN, Ajay, ABBEEL, Pieter. Denoising Diffusion Probabilistic Models. In: *Advances in Neural Information Processing Systems*. 2020, pp. 6840–6851. URL: <https://proceedings.neurips.cc/paper/2020/hash/4c5bcfec8584af0d967f1ab10179ca4b-Abstract.html>.
3. ROMBACH, Robin, BLATTMANN, Andreas, LORENZ, Dominik, ESSER, Patrick, OMMER, Björn. High-Resolution Image Synthesis with Latent Diffusion Models. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2022, pp. 10684–10695. DOI: [10.1109/CVPR52688.2022.01042](https://openaccess.thecvf.com/content/CVPR2022/html/Rombach_High-Resolution_Image_Synthesis_With_Latent_Diffusion_Models_CVPR_2022_paper.html). URL: https://openaccess.thecvf.com/content/CVPR2022/html/Rombach_High-Resolution_Image_Synthesis_With_Latent_Diffusion_Models_CVPR_2022_paper.html.
4. PEEBLES, William, XIE, Saining. *Scalable Diffusion Models with Transformers*. arXiv, 2022. DOI: [10.48550/ARXIV.2212.09748](https://arxiv.org/abs/2212.09748). URL: <https://arxiv.org/abs/2212.09748>.
5. WU, Chenfei, LI, Jiahao, ZHOU, Jingren, LIN, Junyang, GAO, Kaiyuan, YAN, Kun, YIN, Sheng-ming, BAI, Shuai, et al. *Qwen-Image Technical Report*. 2025. URL: <https://arxiv.org/abs/2508.02324>.
6. Z-IMAGE TEAM, CAI, Huanqia, CAO, Sihan, DU, Ruoyi, GAO, Peng, HOI, Steven, HOU, Zhaohui, HUANG, Shijie, et al. *Z-Image: An Efficient Image Generation Foundation Model with Single-Stream Diffusion Transformer*. arXiv, 2025. DOI: [10.48550/ARXIV.2511.22699](https://arxiv.org/abs/2511.22699). URL: <https://arxiv.org/abs/2511.22699>.
7. RODRIGUEZ, Juan A., PURI, Abhay, AGARWAL, Shubham, LARADJI, Issam H., RODRIGUEZ, Pau, RAJESWAR, Sai, VAZQUEZ, David, PAL, Christopher, et al. StarVector: Generating Scalable Vector Graphics Code from Images and Text. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2025, pp. 16175–

16186. DOI: [10.1109/CVPR52734.2025.01508](https://doi.org/10.1109/CVPR52734.2025.01508). URL: <https://arxiv.org/abs/2312.11556>.
8. YANG, Yiying, CHENG, Wei, CHEN, Sijin, ZENG, Xianfang, YIN, Fukun, ZHANG, Jiaxu, WANG, Liao, YU, Gang, et al. *OmnSVG: A Unified Scalable Vector Graphics Generation Model*. 2025. URL: <https://arxiv.org/abs/2504.06263>.
 9. SELINGER, Peter. Potrace: A Polygon-Based Tracing Algorithm. technical report. 2003. URL: <https://potrace.sourceforge.net/potrace.pdf>.
 10. DZIUBA, Maria, JARSKY, Ivan, EFIMOVA, Valeria, FILCHENKOV, Andrey. *Image Vectorization: A Review*. 2023. DOI: [10.48550/arXiv.2306.06441](https://doi.org/10.48550/arXiv.2306.06441). URL: <https://arxiv.org/abs/2306.06441>.
 11. COMPVIS. *Stable Diffusion v1-4 Model Card*. URL: <https://huggingface.co/CompVis/stable-diffusion-v1-4> [ACCESSED 2026-May-18].
 12. SCHUHMANN, Christoph, BEAUMONT, Romain, VENCU, Richard, GORDON, Cade, WIGHTMAN, Ross, CHERTI, Mehdi, COOMBES, Theo, KATTA, Aarush, et al. LAION-5B: An Open Large-Scale Dataset for Training Next Generation Image-Text Models. In: *Advances in Neural Information Processing Systems*. 2022. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/a1859debf3b59d094f3504d5ebb6c25-Abstract-Datasets_and_Benchmarks.html.
 13. WORLD WIDE WEB CONSORTIUM. *Scalable Vector Graphics (SVG) 1.1 (Second Edition): Paths*. 2011. URL: <https://www.w3.org/TR/2011/REC-SVG11-20110816/paths.html>.
 14. BAI, Shuai, CHEN, Keqin, LIU, Xuejing, WANG, Jialin, GE, Wenbin, SONG, Sibao, DANG, Kai, WANG, Peng, et al. *Qwen2.5-VL Technical Report*. 2025. URL: <https://arxiv.org/abs/2502.13923>.
 15. GOOGLE FONTS. *picosvg*. URL: <https://github.com/googlefonts/picosvg> [ACCESSED 2026-May-14].
 16. LIPMAN, Yaron, CHEN, Ricky T. Q., BEN-HAMU, Heli, NICKEL, Maximilian, LE, Matthew. Flow Matching for Generative Modeling. In: *The Eleventh International Conference on Learning Representations*. 2023. URL: <https://openreview.net/forum?id=PqvMRDCJT9t>.

17. LI, Tzu-Mao, LUKÁČ, Michal, GHARBI, Michaël, RAGANKELLEY, Jonathan. Differentiable Vector Graphics Rasterization for Editing and Learning. In: *ACM Transactions on Graphics*. 2020. URL: <https://cseweb.ucsd.edu/~tzli/diffvg/>.
18. CARLIER, Alexandre, DANELLJAN, Martin, ALAHI, Alexandre, TIMOFTE, Radu. DeepSVG: A Hierarchical Generative Network for Vector Graphics Animation. In: *Advances in Neural Information Processing Systems*. 2020. URL: <https://proceedings.neurips.cc/paper/2020/hash/bcf9d6bd14a2095866ce8c950b702341-Abstract.html>.
19. VISION CORTEX RESEARCH GROUP. *VTracer: Raster to Vector Graphics Converter*. URL: <https://github.com/visioncortex/vtracer>.
20. RUIZ, Nataniel, LI, Yuanzhen, JAMPANI, Varun, PRITCH, Yael, RUBINSTEIN, Michael, ABERMAN, Kfir. *DreamBooth: Fine Tuning Text-to-Image Diffusion Models for Subject-Driven Generation*. 2023. URL: <https://arxiv.org/abs/2208.12242>.
21. KUMARI, Nupur, ZHANG, Bingliang, ZHANG, Richard, SHECHTMAN, Eli, ZHU, Jun-Yan. *Multi-Concept Customization of Text-to-Image Diffusion*. 2023. URL: <https://arxiv.org/abs/2212.04488>.
22. SOHN, Kihyuk, RUIZ, Nataniel, LEE, Kimin, CHIN, Daniel Castro, BLOK, Irina, CHANG, Huiwen, BARBER, Jarred, JIANG, Lu, et al. *StyleDrop: Text-to-Image Generation in Any Style*. 2023. URL: <https://arxiv.org/abs/2306.00983>.
23. HU, Edward J., SHEN, Yelong, WALLIS, Phillip, ALLEN-ZHU, Zeyuan, LI, Yuanzhi, WANG, Shean, WANG, Lu, CHEN, Weizhu. LoRA: Low-Rank Adaptation of Large Language Models. In: *ICLR*. OpenReview.net, 2022. URL: <http://dblp.uni-trier.de/db/conf/iclr/iclr2022.html#HuSWALWWC22>.
24. MIKRONAI. *svg-svgrepo*. URL: <https://huggingface.co/datasets/mikronai/svg-svgrepo> [ACCESSED 2026-April-27].
25. SVG REPO LLC. *SVG Repo*. URL: <https://www.svgrepo.com/> [ACCESSED 2026-May-4].
26. BLACK FOREST LABS. *FLUX.2-dev*. URL: <https://huggingface.co/black-forest-labs/FLUX.2-dev> [ACCESSED 2026-May-14].
27. CAO, Siyu, CHEN, Hangting, CHEN, Peng, CHENG, Yiji, CUI, Yutao, DENG, Xinchu, DONG, Ying, GONG, Kipper,

- et al. HunyuanImage 3.0 Technical Report. In: *arXiv preprint arXiv:2509.23951*. 2025.
28. BAFGHI, Reza Akbarian, HARILAL, Nidhin, MONTELEONI, Claire, RAISSI, Maziar. Parameter Efficient Fine-tuning of Self-supervised ViTs without Catastrophic Forgetting. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. 2024, pp. 3679–3684.
 29. HO, Jonathan, SALIMANS, Tim. Classifier-Free Diffusion Guidance. In: *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*. 2021. URL: <https://openreview.net/forum?id=qw8AKxfYbI>.
 30. TONGYI-MAI. *Z-Image-Turbo*. URL: <https://huggingface.co/Tongyi-MAI/Z-Image-Turbo> [ACCESSED 2026-May-14].
 31. GANDIKOTA, Rohit, BAU, David. *Distilling Diversity and Control in Diffusion Models*. arXiv, 2025. DOI: [10.48550/ARXIV.2503.10637](https://arxiv.org/abs/2503.10637). URL: <https://arxiv.org/abs/2503.10637>.
 32. OSTRIS. *AI Toolkit*. URL: <https://github.com/ostris/ai-toolkit> [ACCESSED 2026-May-14].
 33. LOSHCHILOV, Ilya, HUTTER, Frank. Decoupled Weight Decay Regularization. In: *International Conference on Learning Representations*. 2019. URL: <https://openreview.net/forum?id=Bkg6RiCqY7>.
 34. RADFORD, Alec, KIM, Jong Wook, HALLACY, Chris, RAMESH, Aditya, GOH, Gabriel, AGARWAL, Sandhini, SASTRY, Girish, ASKELL, Amanda, et al. Learning Transferable Visual Models From Natural Language Supervision. In: *Proceedings of the 38th International Conference on Machine Learning*. PMLR, 2021, pp. 8748–8763. URL: <https://proceedings.mlr.press/v139/radford21a.html>.
 35. OQUAB, Maxime, DARCET, Timothée, MOUTAKANNI, Théo, VO, Huy V., SZAFRANIEC, Marc, KHALIDOV, Vasil, FERNANDEZ, Pierre, HAZIZA, Daniel, et al. *DINOv2: Learning Robust Visual Features without Supervision*. 2023. DOI: [10.48550/arXiv.2304.07193](https://arxiv.org/abs/2304.07193). URL: <https://arxiv.org/abs/2304.07193>.
 36. POMAX. *A Primer on Bezier Curves*. URL: <https://pomax.github.io/BezierInfo-2/> [ACCESSED 2026-April-29].

37. INKSCAPE PROJECT. *Using the Command Line*. URL: https://wiki.inkscape.org/wiki/Using_the_Command_Line [ACCESSED 2026-April-29].
38. SCORCH. *svgelements*. URL: <https://pypi.org/project/svgelements/> [ACCESSED 2026-May-14].
39. CATMULL, Edwin, ROM, Raphael. A Class of Local Interpolating Splines. In: *Computer Aided Geometric Design*. New York: Academic Press, 1974, pp. 317–326. DOI: [10.1016/B978-0-12-079050-0.50020-5](https://doi.org/10.1016/B978-0-12-079050-0.50020-5).
40. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Lukasz, POLOSUKHIN, Illia. Attention Is All You Need. In: *Advances in Neural Information Processing Systems*. 2017, pp. 5998–6008. URL: <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
41. LIU, Xingchao, GONG, Chengyue, LIU, Qiang. *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. 2022. URL: <https://arxiv.org/abs/2209.03003>.
42. ESSER, Patrick, KULAL, Sumith, BLATTMANN, Andreas, ENTEZARI, Rahim, MÜLLER, Jonas, SAINI, Harry, LEVI, Yam, LORENZ, Dominik, et al. *Scaling Rectified Flow Transformers for High-Resolution Image Synthesis*. 2024. URL: <https://arxiv.org/abs/2403.03206>.
43. SIMÉONI, Oriane, VO, Huy V., SEITZER, Maximilian, BALDASSARRE, Federico, OQUAB, Maxime, JOSE, Cijo, KHALIDOV, Vasil, SZAFRANIEC, Marc, et al. *DINOv3*. arXiv, 2025. DOI: [10.48550/ARXIV.2508.10104](https://arxiv.org/abs/2508.10104). URL: <https://arxiv.org/abs/2508.10104>.
44. SU, Jianlin, AHMED, Murtadha, LU, Yu, PAN, Shengfeng, BO, Wen, LIU, Yunfeng. RoFormer: Enhanced transformer with Rotary Position Embedding. In: *Neurocomputing*. 2024, pp. 127063. DOI: <https://doi.org/10.1016/j.neucom.2023.127063>. URL: <https://www.sciencedirect.com/science/article/pii/S0925231223011864>.
45. BUTCHER, John C. *Numerical Methods for Ordinary Differential Equations*. Chichester: John Wiley & Sons, 2003. ISBN 978-0-471-96758-3. DOI: [10.1002/0470868279](https://doi.org/10.1002/0470868279).

46. DAO, Tri. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. 2023. URL: <https://arxiv.org/abs/2307.08691>.
47. WANG, Zhou, BOVIK, Alan C., SHEIKH, Hamid R., SIMONCELLI, Eero P. Image Quality Assessment: From Error Visibility to Structural Similarity. In: *IEEE Transactions on Image Processing*. 2004, pp. 600–612. doi: [10.1109/TIP.2003.819861](https://doi.org/10.1109/TIP.2003.819861).
48. JACCARD, Paul. Étude comparative de la distribution florale dans une portion des Alpes et du Jura. In: *Bulletin de la Société Vaudoise des Sciences Naturelles*. 1901, pp. 547–579. doi: [10.5169/seals-266450](https://doi.org/10.5169/seals-266450).
49. MARTIN, David R., FOWLKES, Charless C., MALIK, Jitendra. Learning to Detect Natural Image Boundaries Using Local Brightness, Color, and Texture Cues. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2004, pp. 530–549. doi: [10.1109/TPAMI.2004.1273918](https://doi.org/10.1109/TPAMI.2004.1273918).
50. BORGEFORS, Gunilla. Hierarchical Chamfer Matching: A Parametric Edge Matching Algorithm. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 1988, pp. 849–865. doi: [10.1109/34.9107](https://doi.org/10.1109/34.9107).
51. HUTTENLOCHER, Daniel P., KLANDERMAN, Gregory A., RUCKLIDGE, William J. Comparing Images Using the Hausdorff Distance. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 1993, pp. 850–863. doi: [10.1109/34.232073](https://doi.org/10.1109/34.232073).
52. STARVECTOR. *StarVector Models*. URL: <https://huggingface.co/starvector> [ACCESSED 2026-May-18].
53. ZHANG, Lvmin, AGRAWALA, Maneesh. *Transparent Image Layer Diffusion using Latent Transparency*. arXiv, 2024. doi: [10.48550/ARXIV.2402.17113](https://doi.org/10.48550/ARXIV.2402.17113). URL: <https://arxiv.org/abs/2402.17113>.

A Implementation artifacts

The source code for the implementation developed in this thesis is available in the GitHub repository: <https://github.com/JosefKuchar/svg-generator>.

Trained model checkpoints and accompanying model artifacts are available on Hugging Face: <https://huggingface.co/JosefKuchar/svg-generator>.

The converted Bezier dataset used for raster-to-vector fine-tuning is available on Hugging Face: <https://huggingface.co/datasets/JosefKuchar/bezier-dataset>.

Both the implementation and the trained model artifacts are released under the MIT license.